

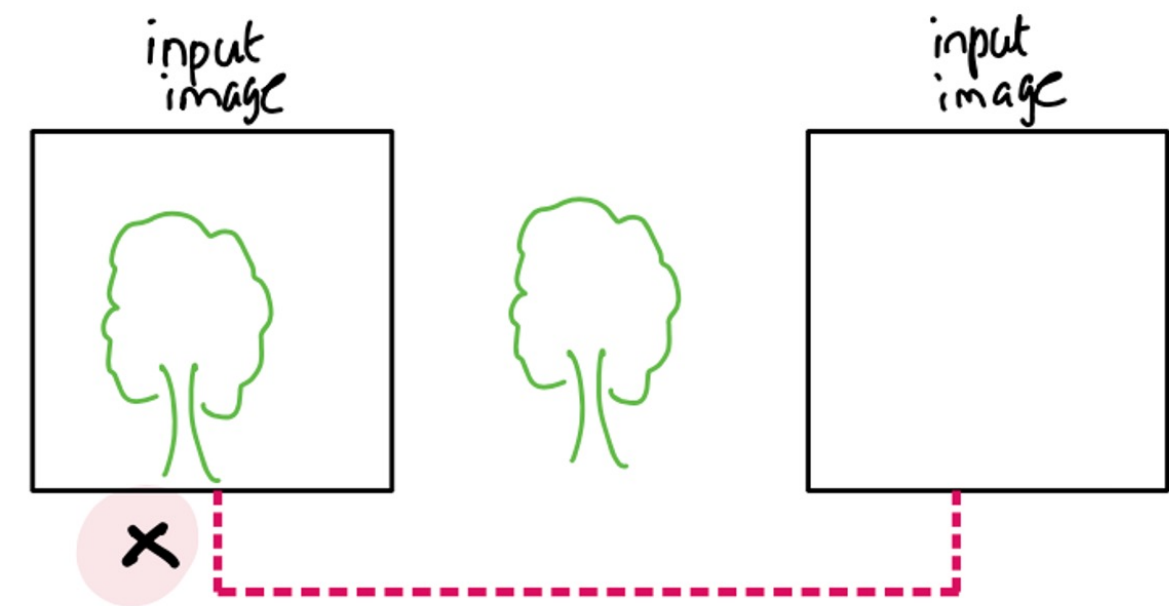
GNN properties: node permutation invariance & equivariance

Node permutation invariance

Reminder: Transformation invariance in Euclidean deep learning

$\mathbf{x}$ is *invariant* to a transformation $\mathbf{t}[\mathbf{x}]$ if:

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{f}[\mathbf{x}].$$

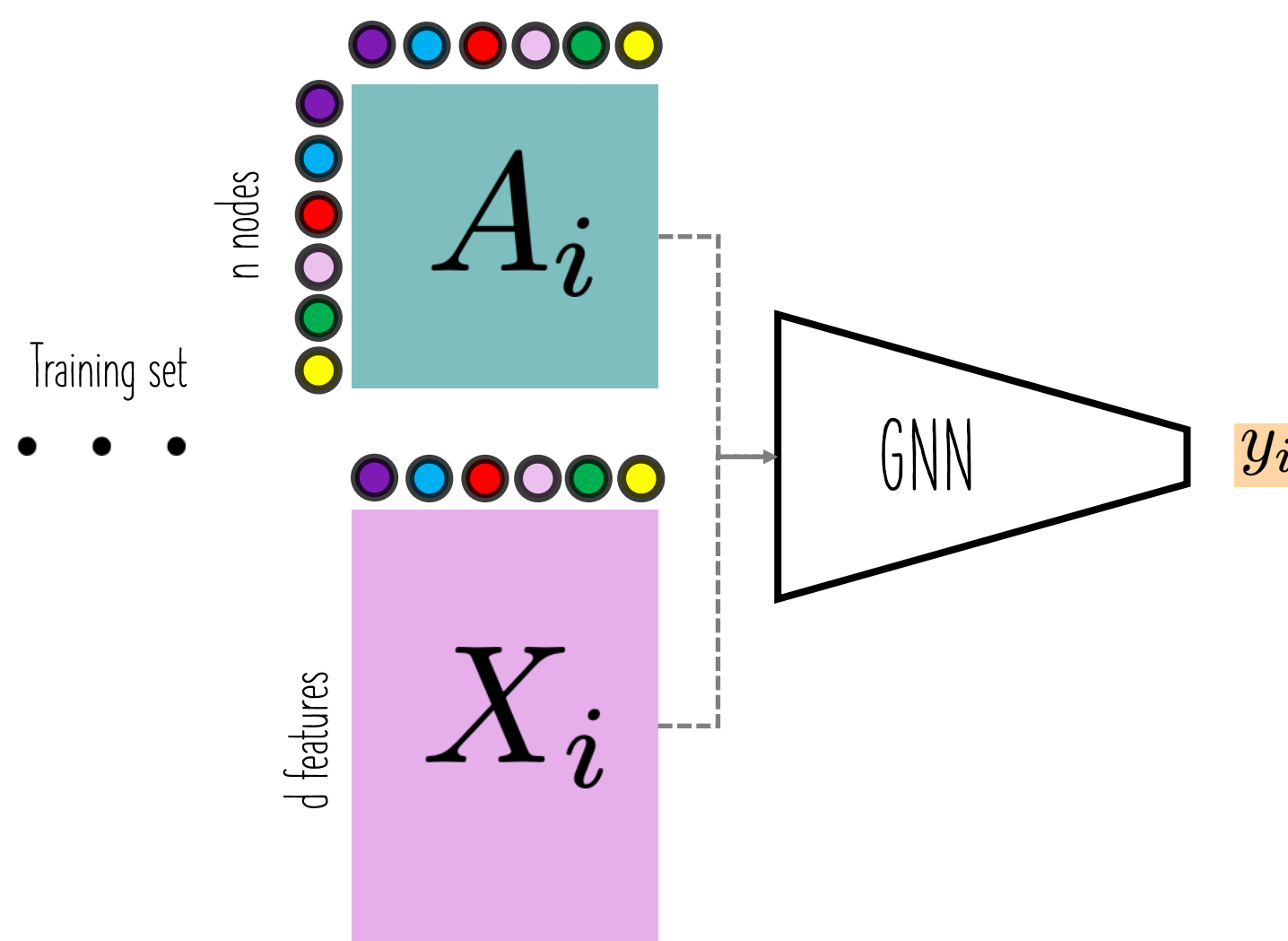


- 1) The output of the function $\mathbf{f}[\mathbf{x}]$ is the same regardless of the transformation $\mathbf{t}[\mathbf{x}]$.
- 2) Networks for image classification should be invariant to geometric transformations of the image.
- 3) The network $\mathbf{f}[\mathbf{x}]$ should identify an image as containing the same object, even if it has been translated, rotated, flipped, or warped.

Which type of data invariance we need to preserve in GNNs and why?

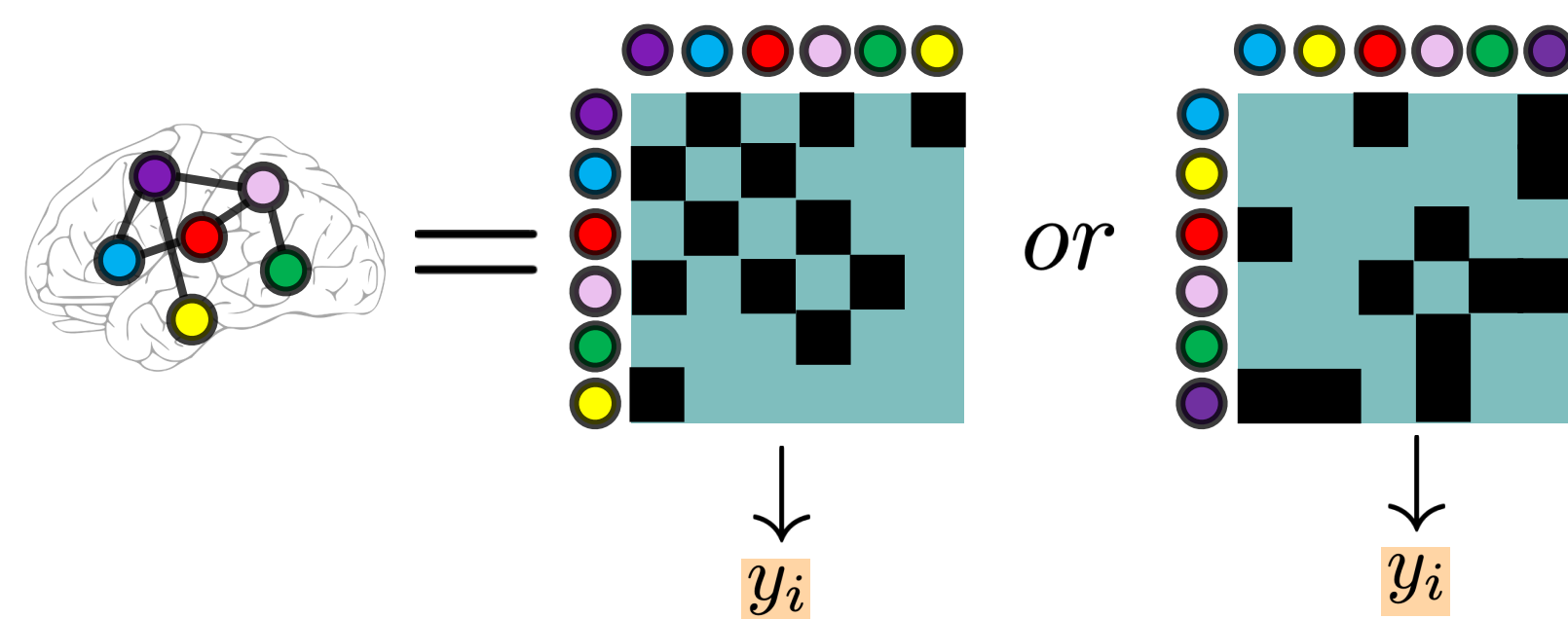
Case study (graph classification)

$$D = \left\{ \begin{array}{c} \text{graph} \\ (G_1, y_1) \end{array}, \dots, \begin{array}{c} \text{graph} \\ (G_n, y_n) \end{array} \right\}$$



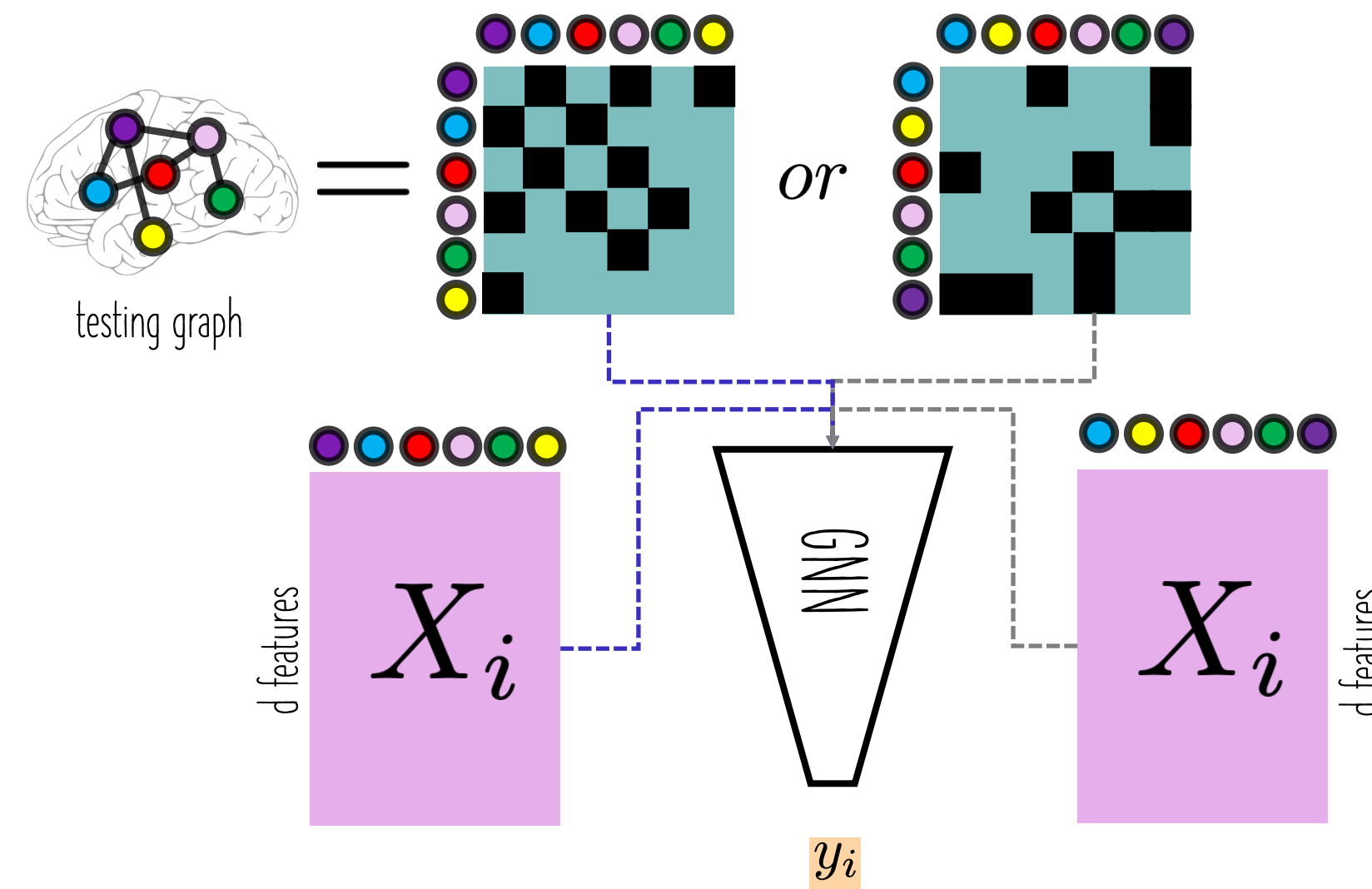
What happens when the GNN model is node order-variant?

If we permute the node indices, the topology of the input graph remains unchanged. We have $n!$ potential matrices representing a graph with n nodes. However, what happens to the predicted output if the GNN model is node order/location variant?



Node permutation invariance in GNNs

GNNs need to be permutation-invariant, thereby learning representations (embeddings) that do not depend on the indexing of the graph nodes.



Permutation invariance in GNNs implies that, for a given node, each GCN layer (including its operations) computing its embedding is permutation invariant.

What does it mean to have the same node embedding regardless of its indexing?

Let $\mathbf{f}$ denote the mapping function of a GNN onto the final d -dimensional embedding space:

$$\begin{aligned} f: \mathcal{G} &\rightarrow \mathbb{R}^d \\ G &\mapsto f(A, X), \end{aligned}$$

Let i and j denote different node orderings of the same graph, $\mathbf{f}$ is a permutation-invariant function if it satisfies the following of any orderings i and j :

$$f(A_i, X_i) = f(A_j, X_j)$$

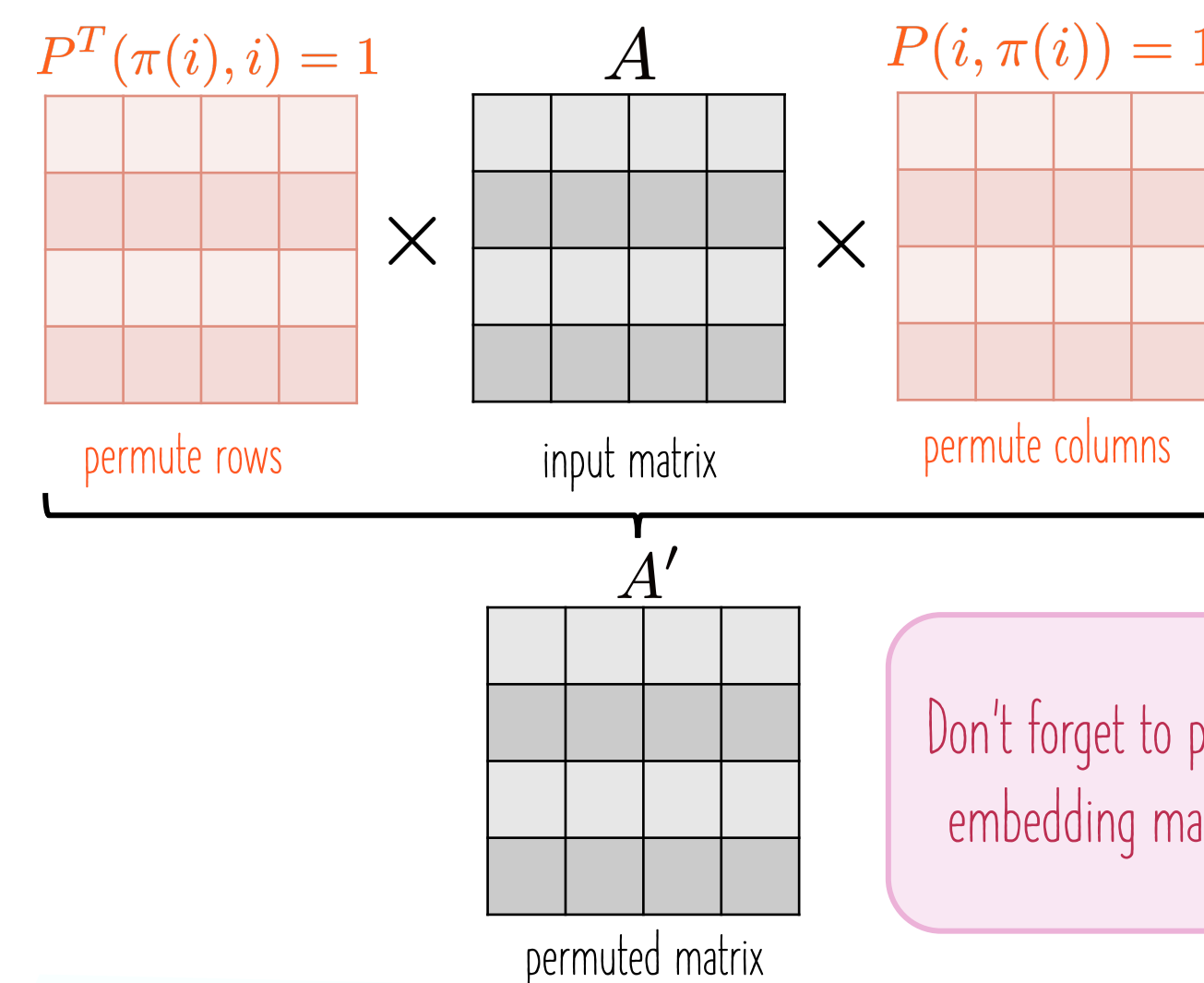
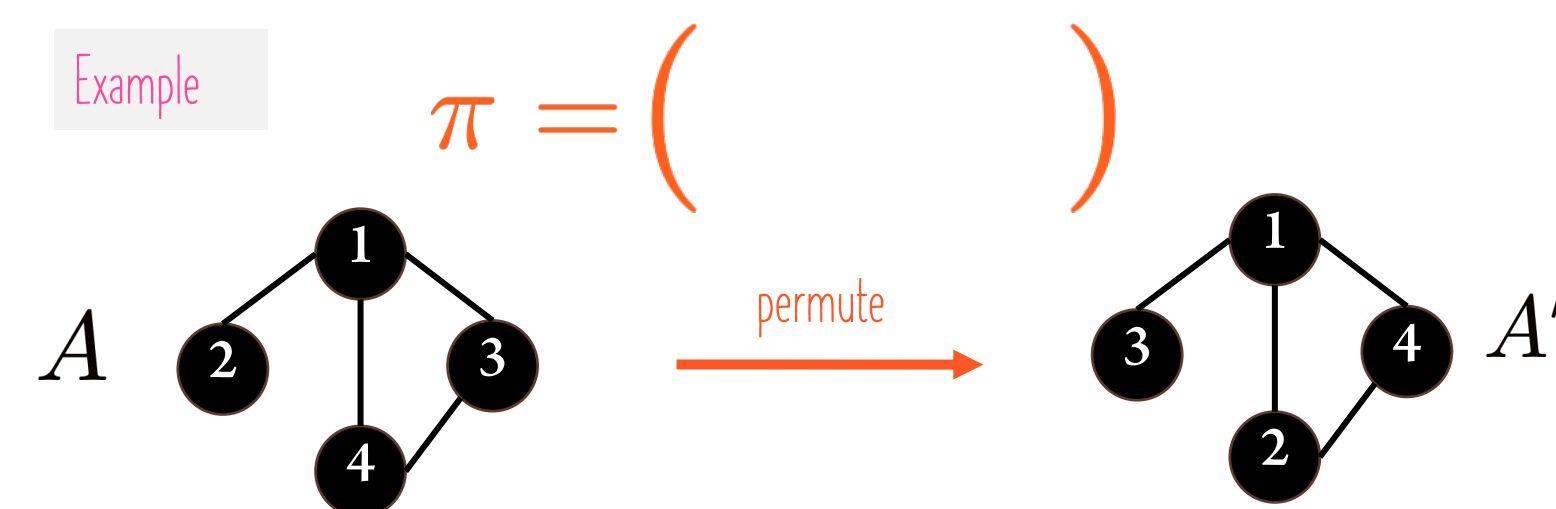
Formalization:

A graph function $\mathbf{f}_k$ is a permutation-invariant if for each GCN layer k , we satisfy the following for any permutation matrix $\mathbf{P}$:

$$\begin{aligned} f: \mathbb{R}^{n \times n} \times \mathbb{R}^{d_k \times n} &\rightarrow \mathbb{R}^{d_{k+1}} \\ f(A, H_k) &= f(\mathbf{P}^T \mathbf{A} \mathbf{P}, H_k \mathbf{P}) \end{aligned}$$

→ The learned graph embedding is invariant to its node permutation.

Example

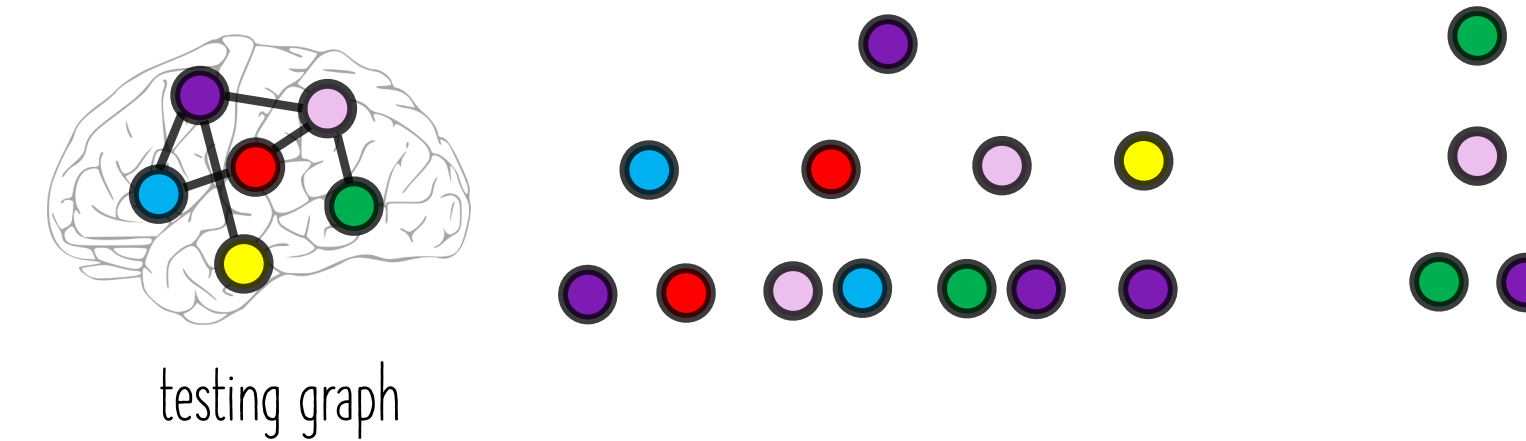


Don't forget to permute the embedding matrix $\mathbf{H}_k$!

$$P(i, \pi(i)) = 1$$

$$\pi = \begin{pmatrix} & & & \\ & & & \\ & & & \\ & & & \end{pmatrix}$$

The node computational graphs using 2 layers



True or false?

1. Permutations π are bijections from a set to itself.
2. Permutation is an associative function: $\pi \circ (\sigma \circ \delta) = (\pi \circ \sigma) \circ \delta$
3. Permutation is a commutative function: $\pi \circ \sigma = \sigma \circ \pi$

- Does the structure of the computational graph of a given node depends on its color (here denoting its index)?

If we permute colors, will we get the same computational graph structure?

→ No, the computational graph is permutation invariant 😊

- Does permutation invariance apply to pixel locations in images or text data?

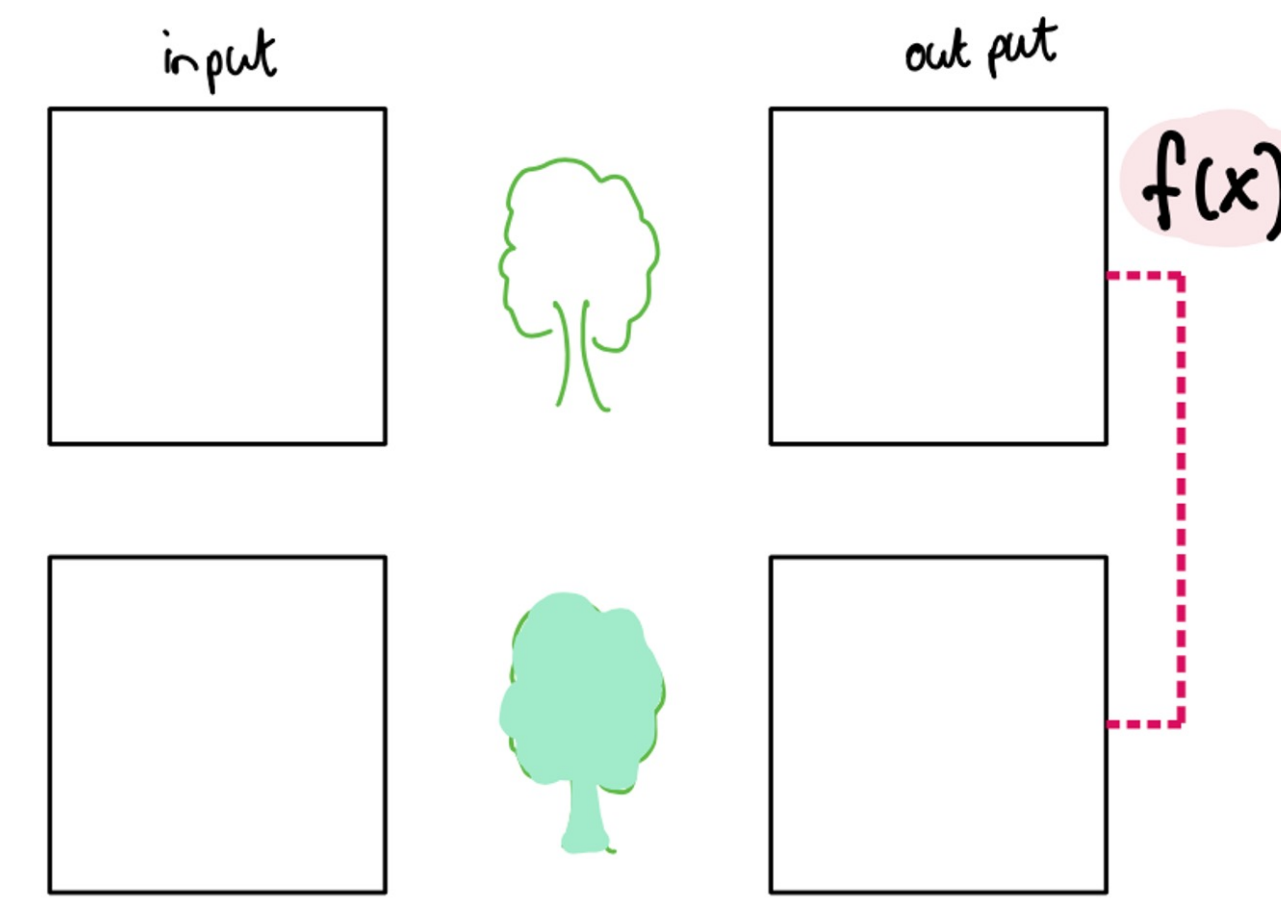
→ No, this is in contrast to images, where permuting the pixels creates a different image, and to text, where permuting the words creates a different sentence.

Node permutation equivariance

Reminder: Transformation equivariance in Euclidean deep learning

A function $\mathbf{f}[\mathbf{x}]$ of an image $\mathbf{x}$ is *equivariant* to a transformation $\mathbf{t}[\mathbf{x}]$ if:

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{t}[\mathbf{f}[\mathbf{x}]].$$



- 1) $\mathbf{f}[\mathbf{x}]$ is equivariant to the transformation $\mathbf{t}[\mathbf{x}]$ if its output changes in the same way under the transformation as the input.
- 2) Networks for per-pixel image segmentation should be equivariant to transformations.
- 3) If the image is translated, rotated, or flipped, the network $\mathbf{f}[\mathbf{x}]$ should return a segmentation that has been transformed in the same way.

Node permutation equivariance in GNNs

Considering all nodes in a graph, GCN computation is permutation equivariant.

→ each layer must be equivariant with respect to permutations of the node indices.

If we permute the node indices, the node embeddings at each stage will be permuted in the same way. In mathematical terms, if $\mathbf{P}$ is a permutation matrix then we must have:

$$\mathbf{H}_{k+1} \mathbf{P} = \mathbf{F}[\mathbf{H}_k \mathbf{P}, \mathbf{P}^T \mathbf{A} \mathbf{P}, \phi_k]$$

For node classification and edge prediction tasks, the output should also be equivariant with respect to permutations of the node indices.

For graph-level tasks, the final layer aggregates information from across the graph so the output is invariant to the order of the nodes. In fact, the output layer achieves the following for any permutation matrix $\mathbf{P}$:

$$\begin{aligned} y &= \text{sig}[\beta_K + \omega_K \mathbf{H}_K \mathbf{1} / N] \\ &= \text{sig}[\beta_K + \omega_K \mathbf{H}_K \mathbf{P} \mathbf{1} / N] \end{aligned}$$

😊 For graphs, it is possible to define networks that ensure equivariance or invariance to permutations.

Problem solving

Problem 13.7 Consider the simple GNN layer:

$$\begin{aligned} \mathbf{H}_{k+1} &= \text{GraphLayer}[\mathbf{H}_k, \mathbf{A}] \\ &= \mathbf{a} [\beta_k \mathbf{1}^T + \Omega_k [\mathbf{H}_k; \mathbf{H}_k \mathbf{A}]], \end{aligned} \quad (13.28)$$

where $\mathbf{H}$ is a $D \times N$ matrix containing the N node embeddings in its columns, $\mathbf{A}$ is the $N \times N$ adjacency matrix, β is the bias vector and Ω is the weight matrix. The notation $[\mathbf{H}_k; \mathbf{H}_k \mathbf{A}]$ indicates vertical concatenation of the matrices $\mathbf{H}_k$ and $\mathbf{H}_k \mathbf{A}$. Show that this layer is equivariant to permutations of the node order, so that:

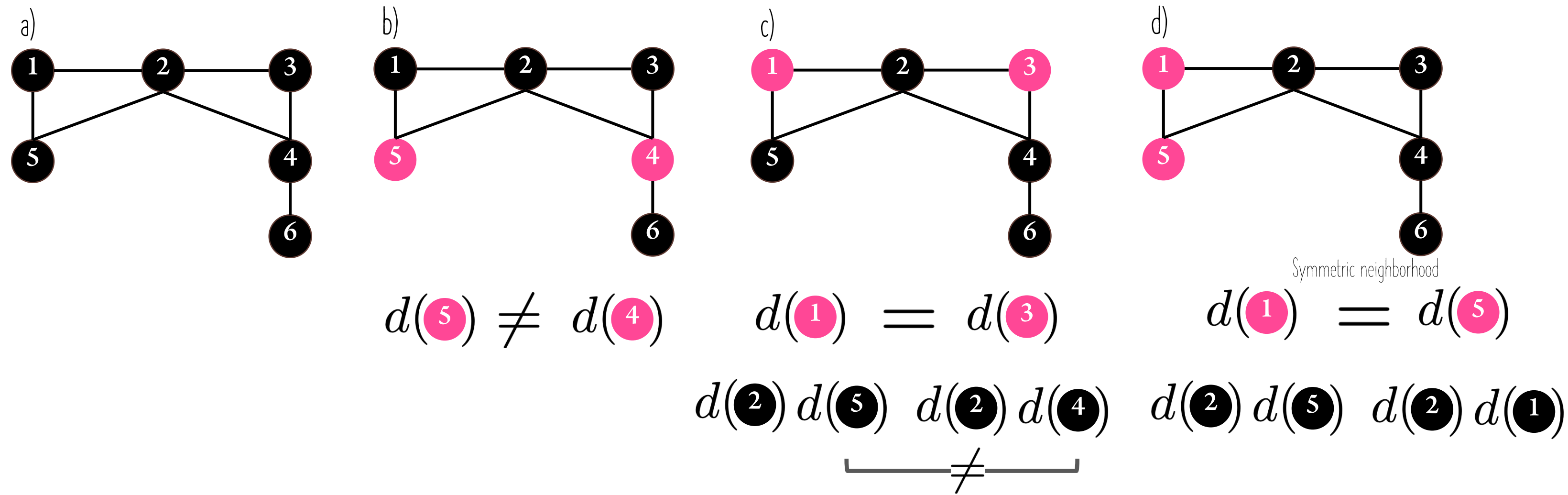
$$\text{GraphLayer}[\mathbf{H}_k, \mathbf{A}] \mathbf{P} = \text{GraphLayer}[\mathbf{H}_k \mathbf{P}, \mathbf{P}^T \mathbf{A} \mathbf{P}], \quad (13.29)$$

where $\mathbf{P}$ is an $N \times N$ permutation matrix.

$$\begin{aligned} \text{GraphLayer}[\mathbf{H}_k \mathbf{P}, \mathbf{P}^T \mathbf{A} \mathbf{P}] &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \begin{bmatrix} \mathbf{H}_k \mathbf{P} \\ \mathbf{H}_k \mathbf{P} \mathbf{P}^T \mathbf{A} \mathbf{P} \end{bmatrix} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \begin{bmatrix} \mathbf{H}_k \mathbf{P} \\ \mathbf{H}_k \mathbf{A} \mathbf{P} \end{bmatrix} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T \mathbf{P} + \Omega_k \begin{bmatrix} \mathbf{H}_k \mathbf{P} \\ \mathbf{H}_k \mathbf{A} \mathbf{P} \end{bmatrix} \right] \mathbf{P} \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \begin{bmatrix} \mathbf{H}_k \\ \mathbf{H}_k \mathbf{A} \end{bmatrix} \right] \mathbf{P} \\ &= \text{GraphLayer}[\mathbf{H}_k, \mathbf{A}] \mathbf{P} \end{aligned}$$

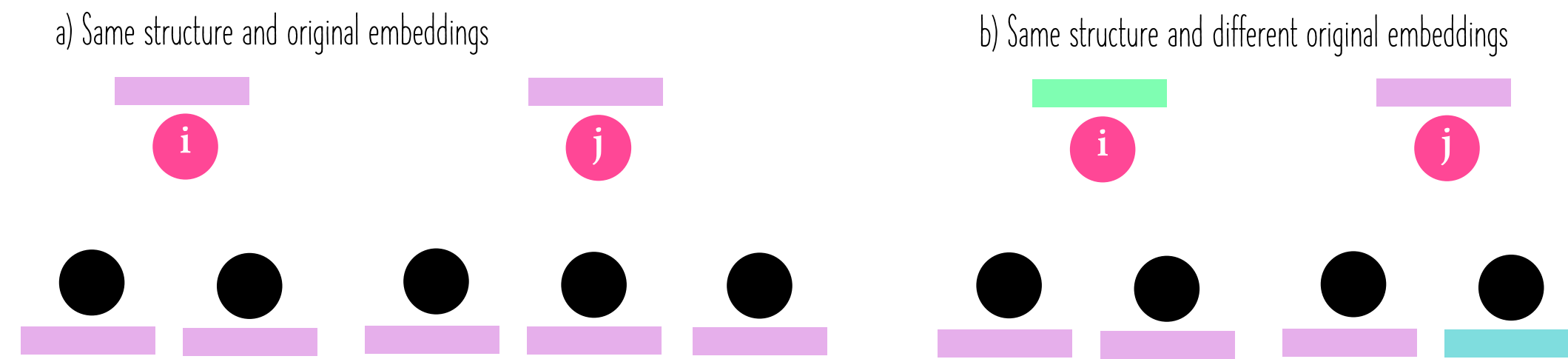
GNN properties: expressive power (expressiveness)

K-hop similarity in node local structure



Expressive power of a GNN model

Definition: An **expressive** GNN model maximizes its **node representation capacity** in the embedding space by mapping **different** local neighborhoods to **different** embeddings (particularly when we have nodes with similar embeddings).



Mathematically, a highly expressive GNN maps local neighborhoods to node embeddings **injectively**.

→ It uses an **injective neighbor aggregation** at each layer k .

→ Since the neighborhood aggregation is modeled through **the node computational graph**, we can define an expressive GNN as a model that maps nodes with **different** computation graphs to different **embeddings**.

For two nodes n and m have two different computational graphs, then their embeddings should be different **at each layer** $k+1$:

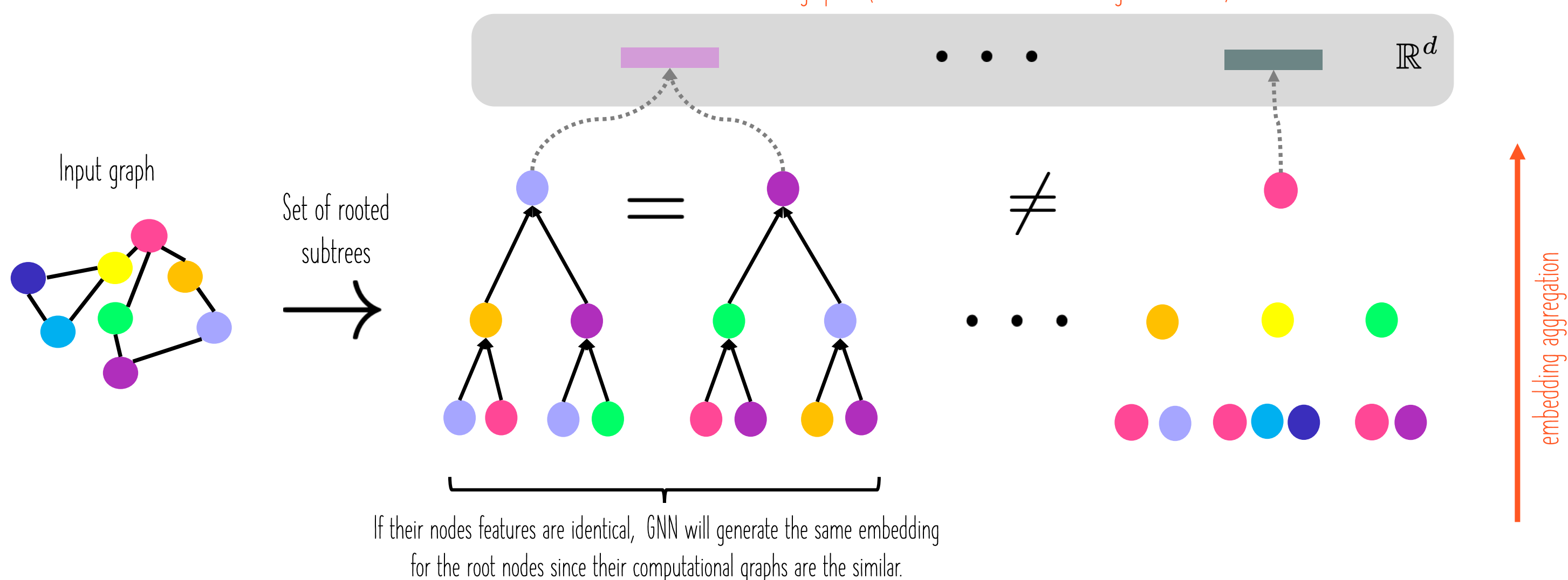
$$\text{If } C(n) \neq C(m) \text{ then } \mathbf{h}_{k+1}^{(n)} \neq \mathbf{h}_{k+1}^{(m)}$$

$$\mathbf{h}_{k+1}^{(n)} = \mathbf{a} \left[\beta_k + \Omega_k \cdot \mathbf{h}_k^{(n)} + \Omega_k \cdot \text{agg}[n, k] \right]$$

Rooted subtrees mapping

Computational graphs are identical to **rooted subtree structures** around each node, which are defined by recursively unfolding neighboring nodes from the root nodes.

embedding space (nodes here have identical original features)



Rooted subtree isomorphism test

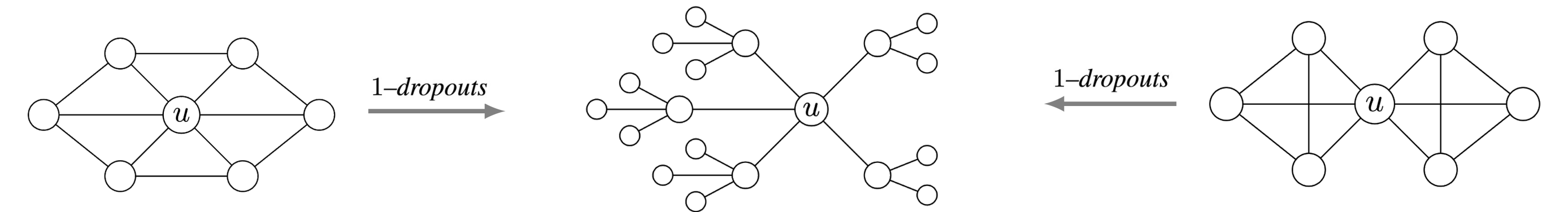
Can GNN node embeddings distinguish different node embeddings neighborhood structure?

→ Yes, if they have been produced using an **injective** aggregation function in each layer.

Is there a case where GNNs can fail to distinguish different structures?

→ Yes, for instance when two different graph structures produce the same computational graph

1) DropGNN (2021)



2) GIN (2019)

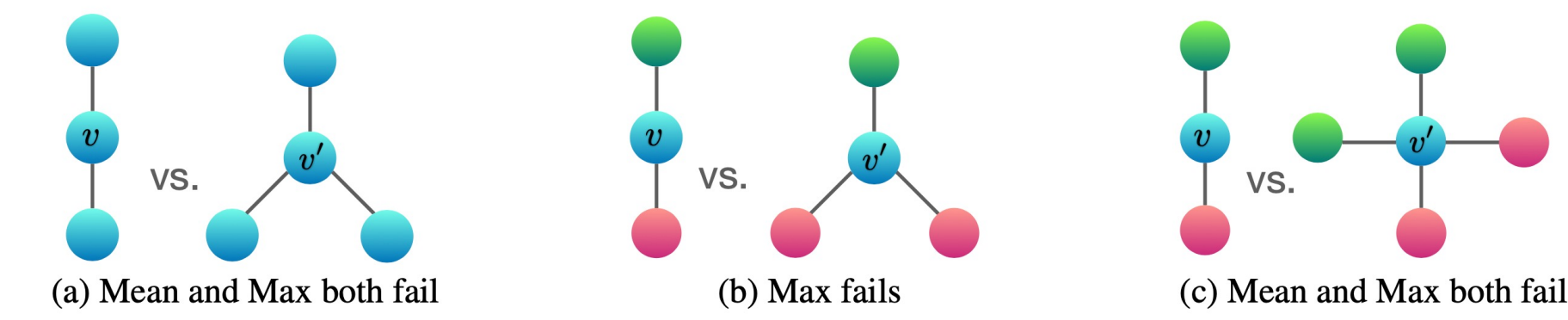


Figure 3: **Examples of graph structures that mean and max aggregators fail to distinguish.** Between the two graphs, nodes v and v' get the same embedding even though their corresponding graph structures differ. Figure 2 gives reasoning about how different aggregators “compress” different multisets and thus fail to distinguish them.

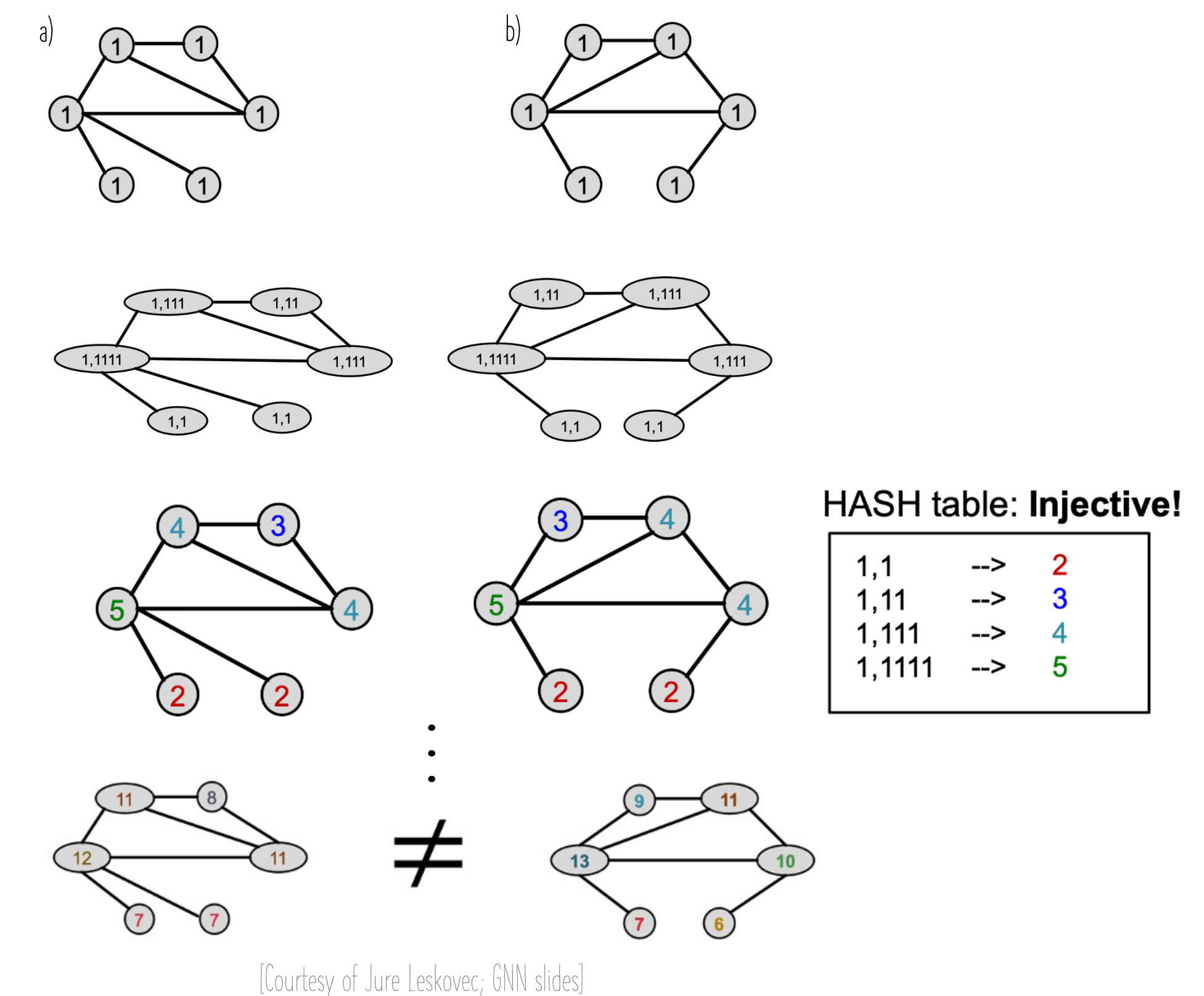
Weisfeiler-Lehman graph isomorphism test

The **graph isomorphism** problem asks whether two graphs are **topologically identical**. This is a challenging problem: no polynomial-time algorithm is known for it yet. The **Weisfeiler-Lehman (WL) test** of graph isomorphism (Weisfeiler & Lehman, 1968) is an effective and computationally efficient test that distinguishes a broad class of graphs.

→ **Goal**: develop a simple architecture that is provably the most expressive among the class of GNNs and is **as powerful as the Weisfeiler-Lehman graph isomorphism test**.

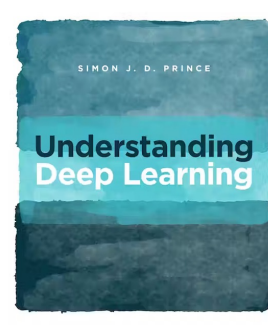
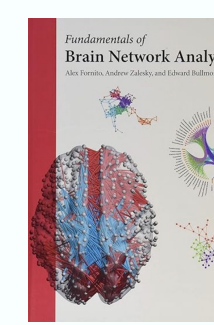
WL color refinement algorithm:

- Assign an initial color $\mathbf{c}^0(\mathbf{v})$ to each node $\mathbf{v}$.
- Iteratively refine node color by: $\mathbf{c}^{k+1}(\mathbf{v}) = \text{HASH}(\mathbf{c}^k(\mathbf{v}), \{\mathbf{c}^k(\mathbf{u})\}_{\mathbf{u} \in \text{ne}[\mathbf{v}]})$, where **HASH** maps different inputs to different colors.
- After K steps of color refinement, $\mathbf{c}^k(\mathbf{v})$ summarizes the structure of K -hop neighborhood.



References:

- DropGNN: Papp PA, Martinkus K, Faber L, Wattenhofer R. DropGNN: Random dropouts increase the expressiveness of graph neural networks. *Advances in Neural Information Processing Systems*. 2021;
- GIN: Xu K, Hu W, Leskovec J, Jegelka S. *How powerful are graph neural networks?* ICLR 2019;
- WL test: Boris Weisfeiler and AA Lehman. *A reduction of a graph to a canonical form and an algebra arising during this reduction*. Nauchno-Technicheskaya Informatsia, 2(9):12-16, 1968.



GNN properties: expressive power (expressiveness)

Graph Isomorphism Network -GIN- (Xu et al., ICLR 2019)

Recall box: summarize what you remember from this lecture below

HOW POWERFUL ARE GRAPH NEURAL NETWORKS?

Keyulu Xu *†
MIT
keyulu@mit.edu

Weihua Hu *†
Stanford University
weihuahu@stanford.edu

Jure Leskovec
Stanford University
jure@cs.stanford.edu

Stefanie Jegelka
MIT
stefje@mit.edu

Key results

Our main results are summarized as follows:

- 1) We show that GNNs are *at most* as powerful as the WL test in distinguishing graph structures.
- 2) We establish conditions on the neighbor aggregation and graph readout functions under which the resulting GNN is *as powerful as* the WL test.
- 3) We identify graph structures that cannot be distinguished by popular GNN variants, such as GCN (Kipf & Welling, 2017) and GraphSAGE (Hamilton et al., 2017a), and we precisely characterize the kinds of graph structures such GNN-based models can capture.
- 4) We develop a simple neural architecture, *Graph Isomorphism Network (GIN)*, and show that its discriminative/representational power is equal to the power of the WL test.

Important theorem

Theorem 3. Let $\mathcal{A} : \mathcal{G} \rightarrow \mathbb{R}^d$ be a GNN. With a sufficient number of GNN layers, $\mathcal{A}$ maps any graphs G_1 and G_2 that the Weisfeiler-Lehman test of isomorphism decides as non-isomorphic, to different embeddings if the following conditions hold:

a) $\mathcal{A}$ aggregates and updates node features iteratively with

$$h_v^{(k)} = \phi \left(h_v^{(k-1)}, f \left(\left\{ h_u^{(k-1)} : u \in \mathcal{N}(v) \right\} \right) \right),$$

where the functions f , which operates on multisets, and ϕ are injective.

b) $\mathcal{A}$'s graph-level readout, which operates on the multiset of node features $\{h_v^{(k)}\}$, is injective.

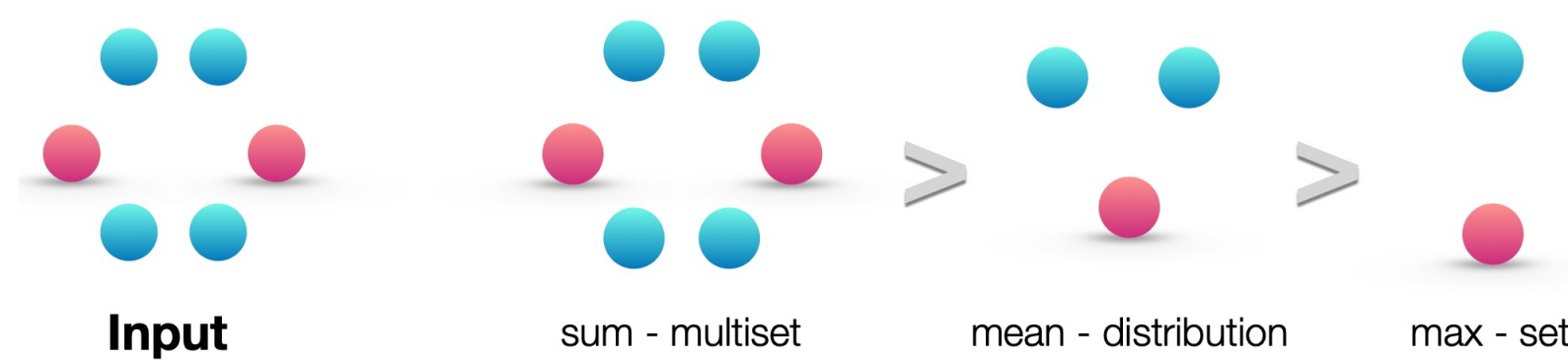


Figure 2: **Ranking by expressive power for sum, mean and max aggregators over a multiset.** Left panel shows the input multiset, *i.e.*, the network neighborhood to be aggregated. The next three panels illustrate the aspects of the multiset a given aggregator is able to capture: sum captures the full multiset, mean captures the proportion/distribution of elements of a given type, and the max aggregator ignores multiplicities (reduces the multiset to a simple set).

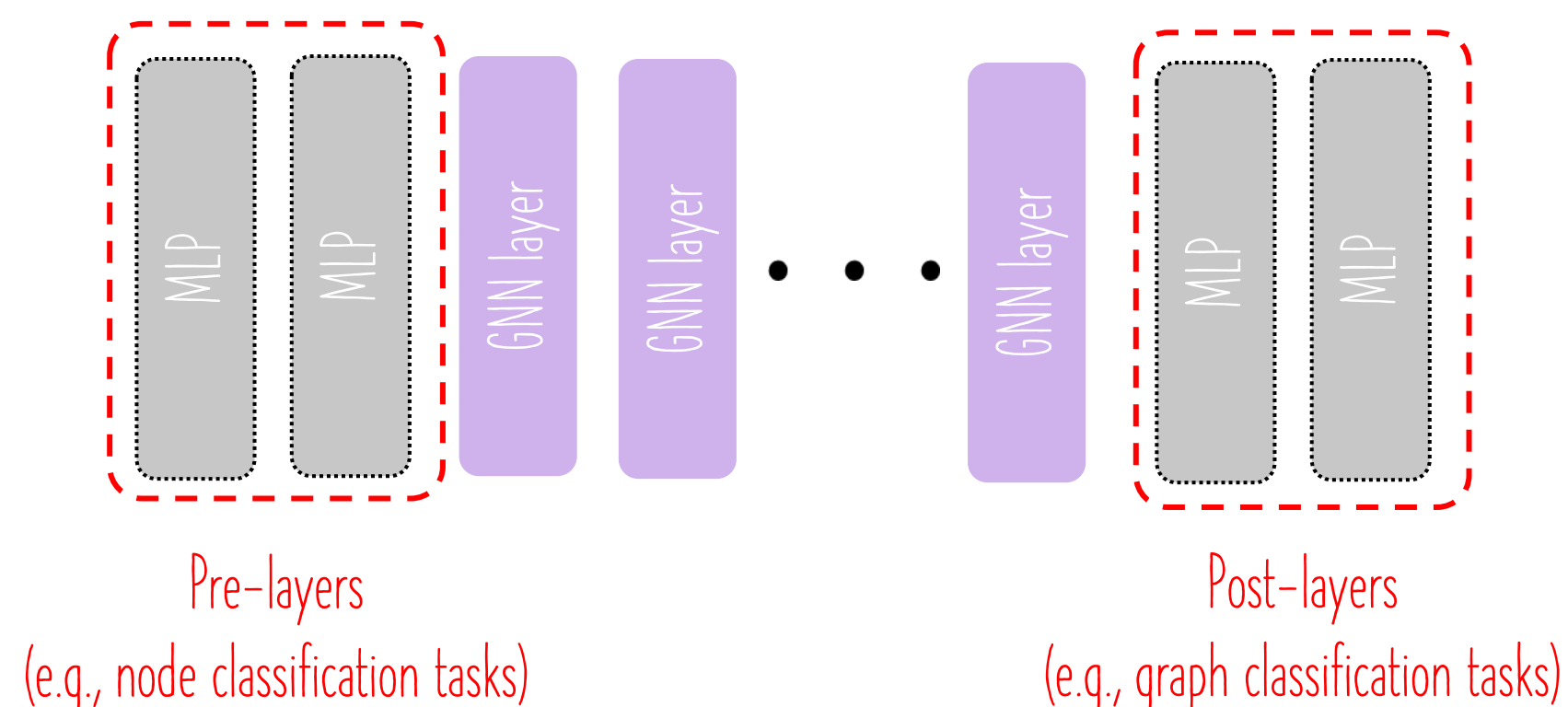
How to maximize the expressiveness of a GNN model?

Multi-layer perceptrons (MLPs) are universal function approximators $\rightarrow$ use them to model and learn f and ϕ .

GIN is a maximally expressive (powerful) GNN among others that may exist (Xu et al., 2019). Its message update rule takes the following form:

$$h_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right)$$

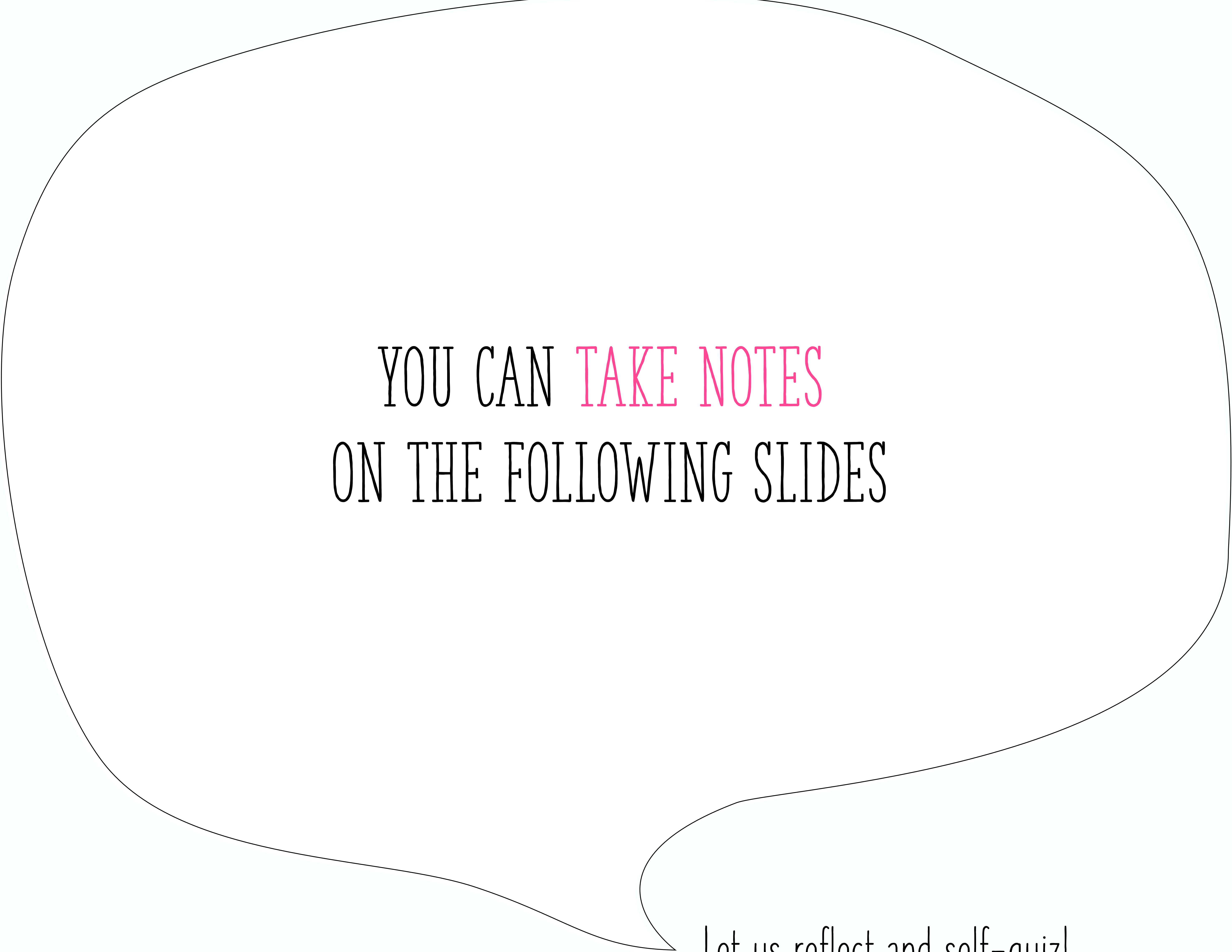
An **expressive** GNN block can take the following form:



References:

1. DropGNN: Papp PA, Martinkus K, Faber L, Wattenhofer R. DropGNN: Random dropouts increase the expressiveness of graph neural networks. *Advances in Neural Information Processing Systems*. 2021;
2. GIN: Xu K, Hu W, Leskovec J, Jegelka S. How powerful are graph neural networks? ICLR 2019;
3. WL test: Boris Weisfeiler and AA Lehman. *A reduction of a graph to a canonical form and an algebra arising during this reduction*. Nauchno-Tekhnicheskaya Informatsia, 2(9):12-16, 1968.





YOU CAN TAKE NOTES
ON THE FOLLOWING SLIDES

Let us reflect and self-quiz!

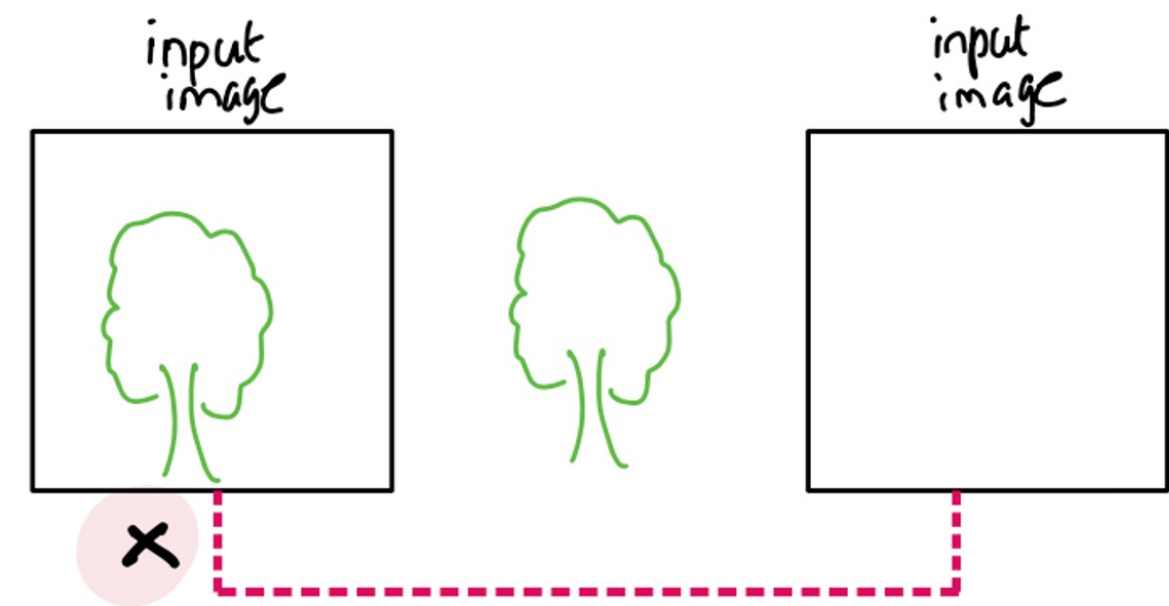
GNN properties: node permutation invariance & equivariance

Node permutation invariance

Reminder: Transformation invariance in Euclidean deep learning

$\mathbf{x}$ is *invariant* to a transformation $\mathbf{t}[\mathbf{x}]$ if:

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{f}[\mathbf{x}].$$

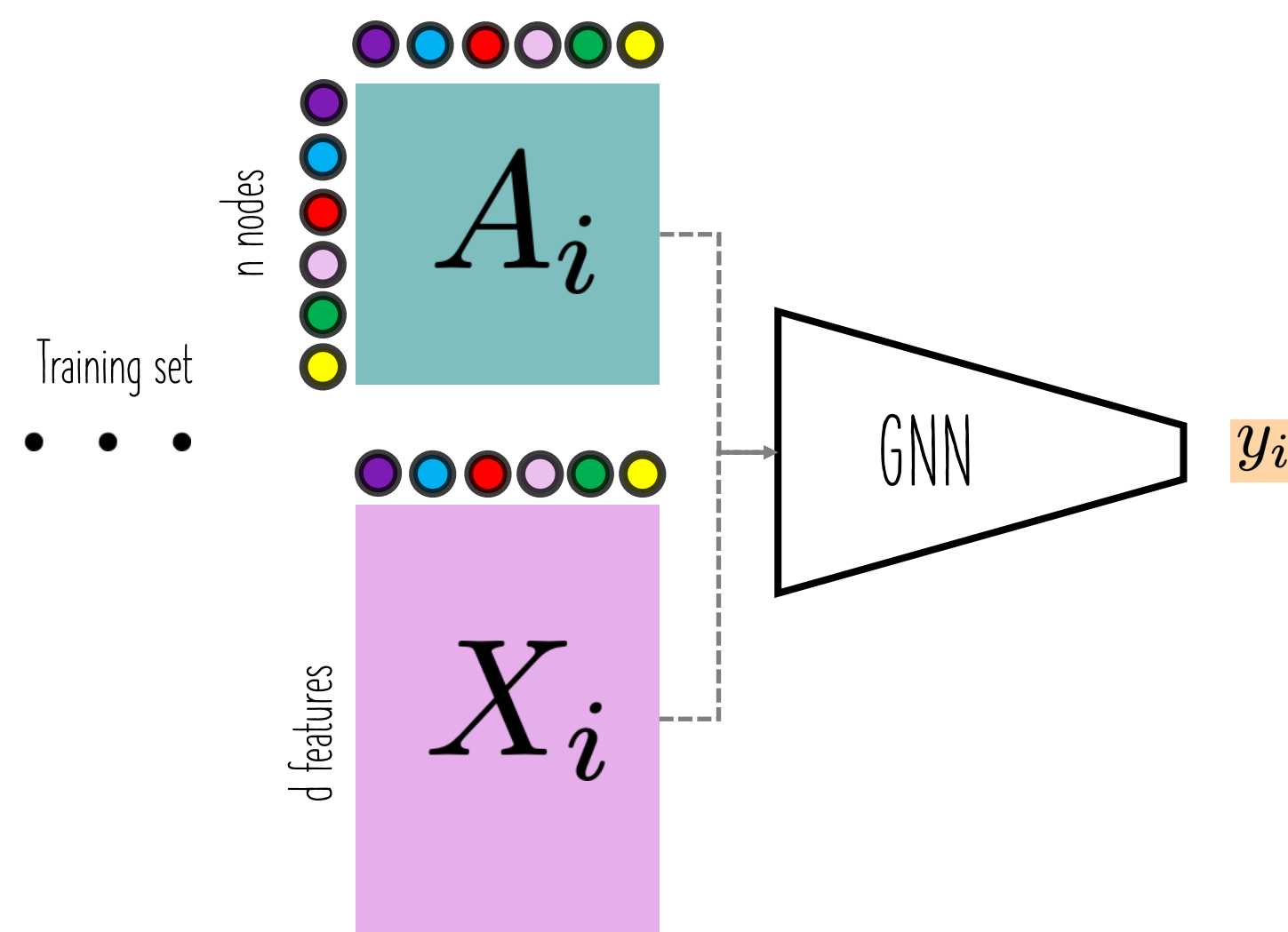


- 1) The output of the function $\mathbf{f}[\mathbf{x}]$ is the same regardless of the transformation $\mathbf{t}[\mathbf{x}]$.
- 2) Networks for image classification should be invariant to geometric transformations of the image.
- 3) The network $\mathbf{f}[\mathbf{x}]$ should identify an image as containing the same object, even if it has been translated, rotated, flipped, or warped.

Which type of data invariance we need to preserve in GNNs and why?

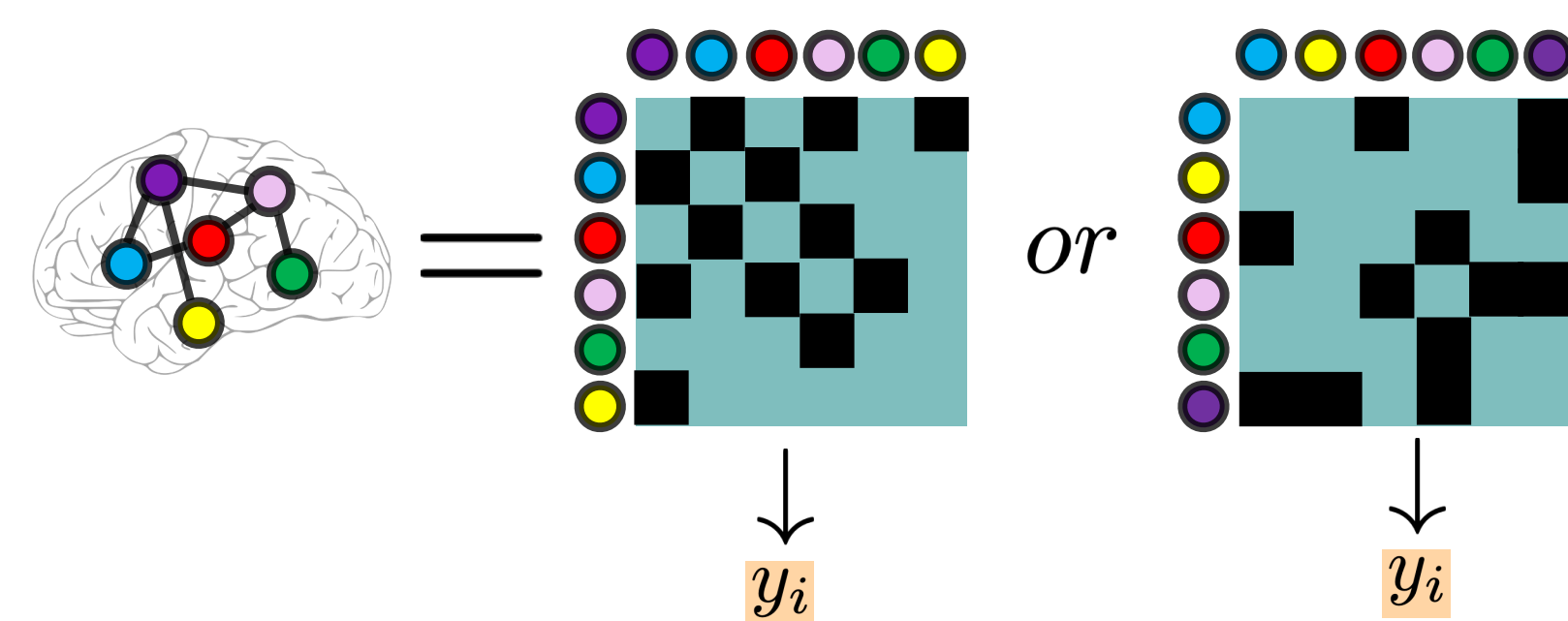
Case study (graph classification)

$$D = \left\{ \begin{array}{c} \text{graph} \\ (G_1, y_1) \end{array}, \dots, \begin{array}{c} \text{graph} \\ (G_n, y_n) \end{array} \right\}$$



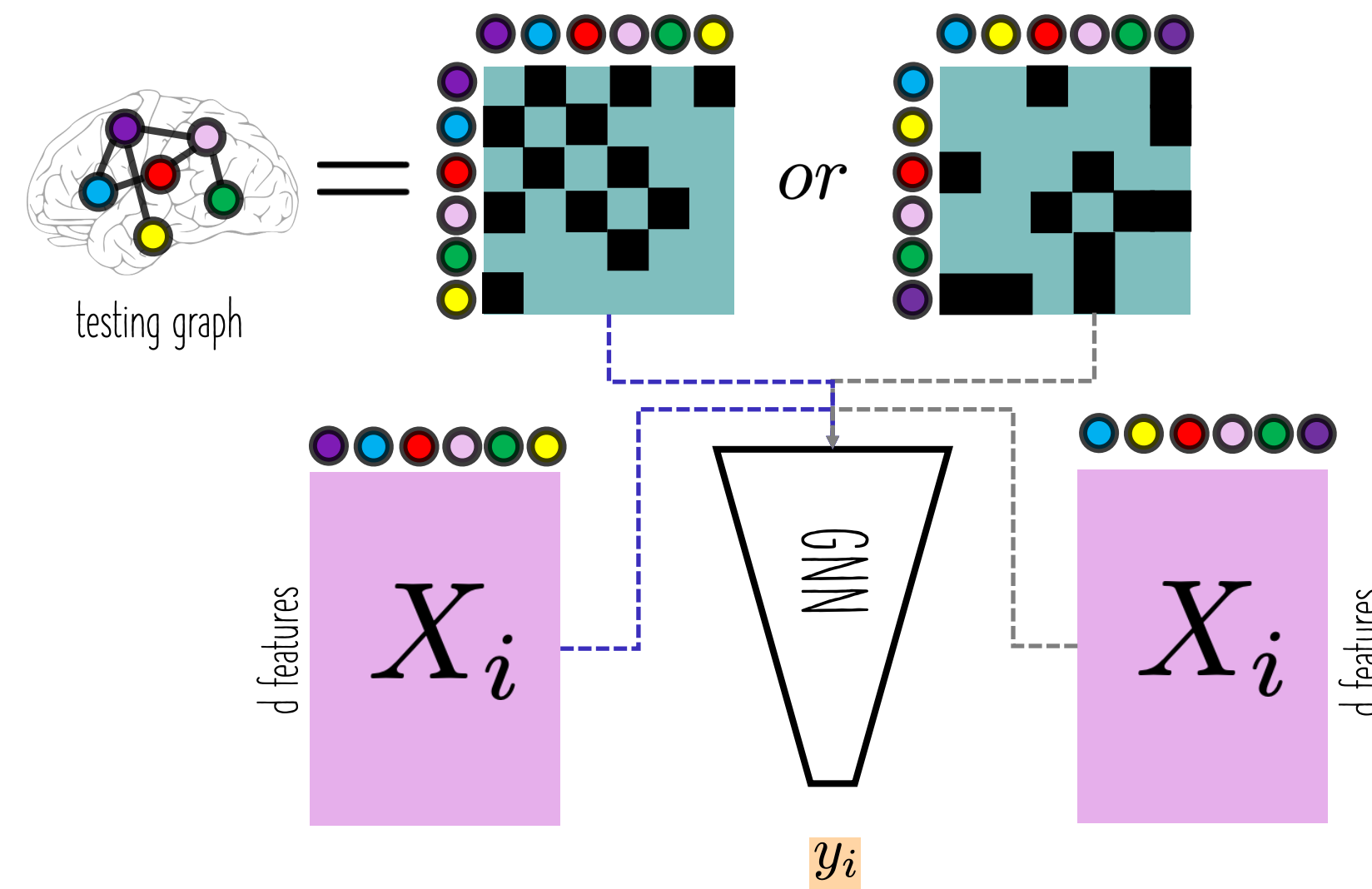
What happens when the GNN model is node order-variant?

If we permute the node indices, the topology of the input graph remains unchanged. We have $n!$ potential matrices representing a graph with n nodes. However, what happens to the predicted output if the GNN model is node order/location variant?



Node permutation invariance in GNNs

GNNs need to be permutation-invariant, thereby learning representations (embeddings) that do not depend on the indexing of the graph nodes.



Permutation invariance in GNNs implies that, for a given node, each GCN layer (including its operations) computing its embedding is permutation invariant.

What does it mean to have the same node imbedding regardless of its indexing?

Let $\mathbf{f}$ denote the mapping function of a GNN onto the final d -dimensional embedding space:

$$\begin{aligned} \mathbf{f}: \mathcal{G} &\longrightarrow \mathbb{R}^d \\ G &\longmapsto \mathbf{f}(A, X), \end{aligned}$$

Let i and j denote different node orderings of the same graph, $\mathbf{f}$ is a permutation-invariant function if it satisfies the following of any orderings i and j :

$$\mathbf{f}(A_i, X_i) = \mathbf{f}(A_j, X_j)$$

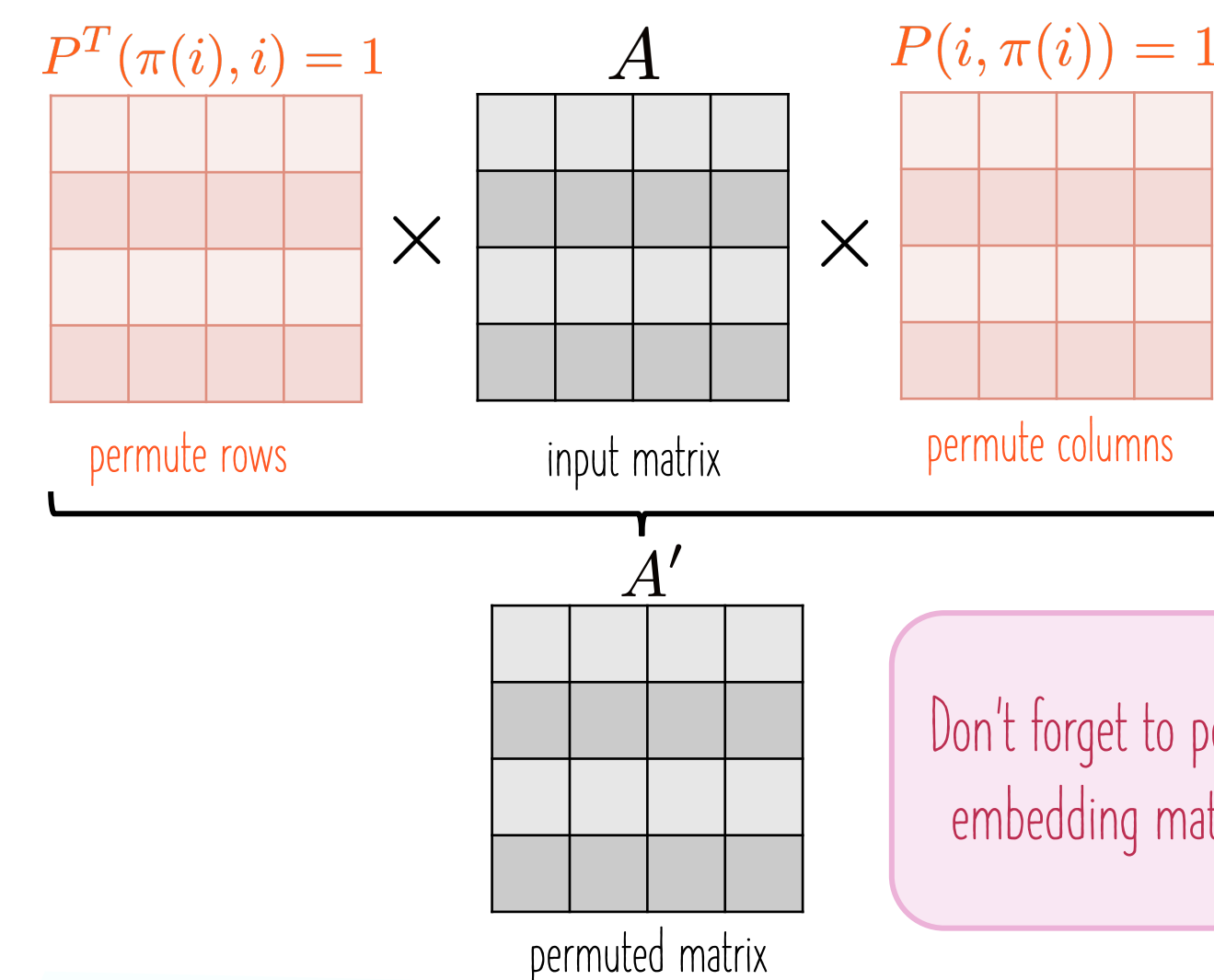
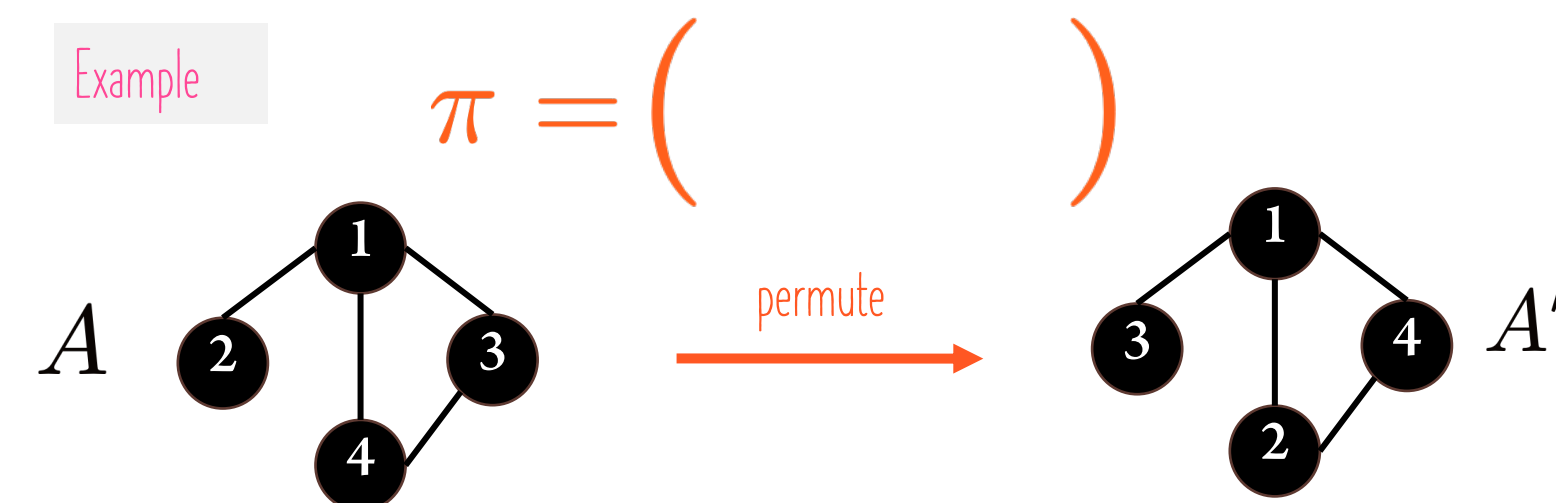
Formalization:

A graph function $\mathbf{f}_k$ is a permutation-invariant if for each GCN layer k , we satisfy the following for any permutation matrix $\mathbf{P}$:

$$\begin{aligned} \mathbf{f}: \mathbb{R}^{n \times n} \times \mathbb{R}^{d_k \times n} &\longrightarrow \mathbb{R}^{d_{k+1}} \\ \mathbf{f}(A, H_k) &= \mathbf{f}(\mathbf{P}^T A \mathbf{P}, H_k \mathbf{P}) \end{aligned}$$

→ The learned graph embedding is invariant to its node permutation.

Example



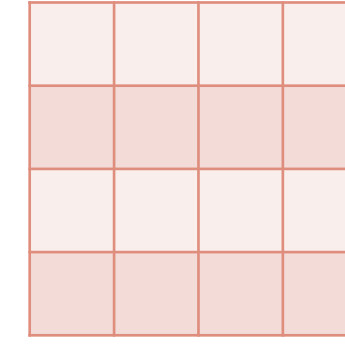
Don't forget to permute the embedding matrix $\mathbf{H}_k$!

Personal reflections and notes

GNN properties: node permutation invariance & equivariance

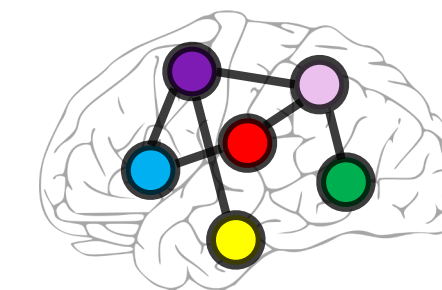
Personal reflections and notes

$$P(i, \pi(i)) = 1$$

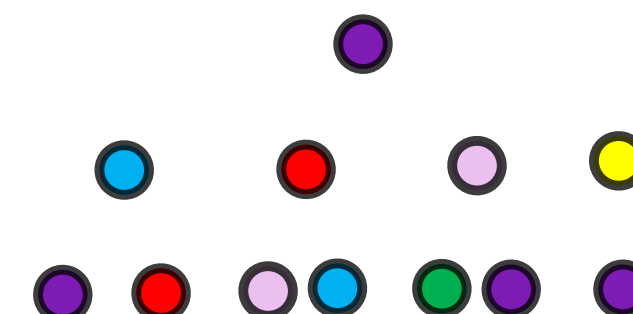


$$\pi = \begin{pmatrix} & & & \\ & & & \\ & & & \\ & & & \end{pmatrix}$$

The node computational graphs using 2 layers



testing graph



- Does the structure of the computational graph of a given node depends on its color (here denoting its index)?

If we permute colors, will we get the same computational graph structure?

→ No, the computational graph is permutation invariant 😊

- Does permutation invariance apply to pixel locations in images or text data?

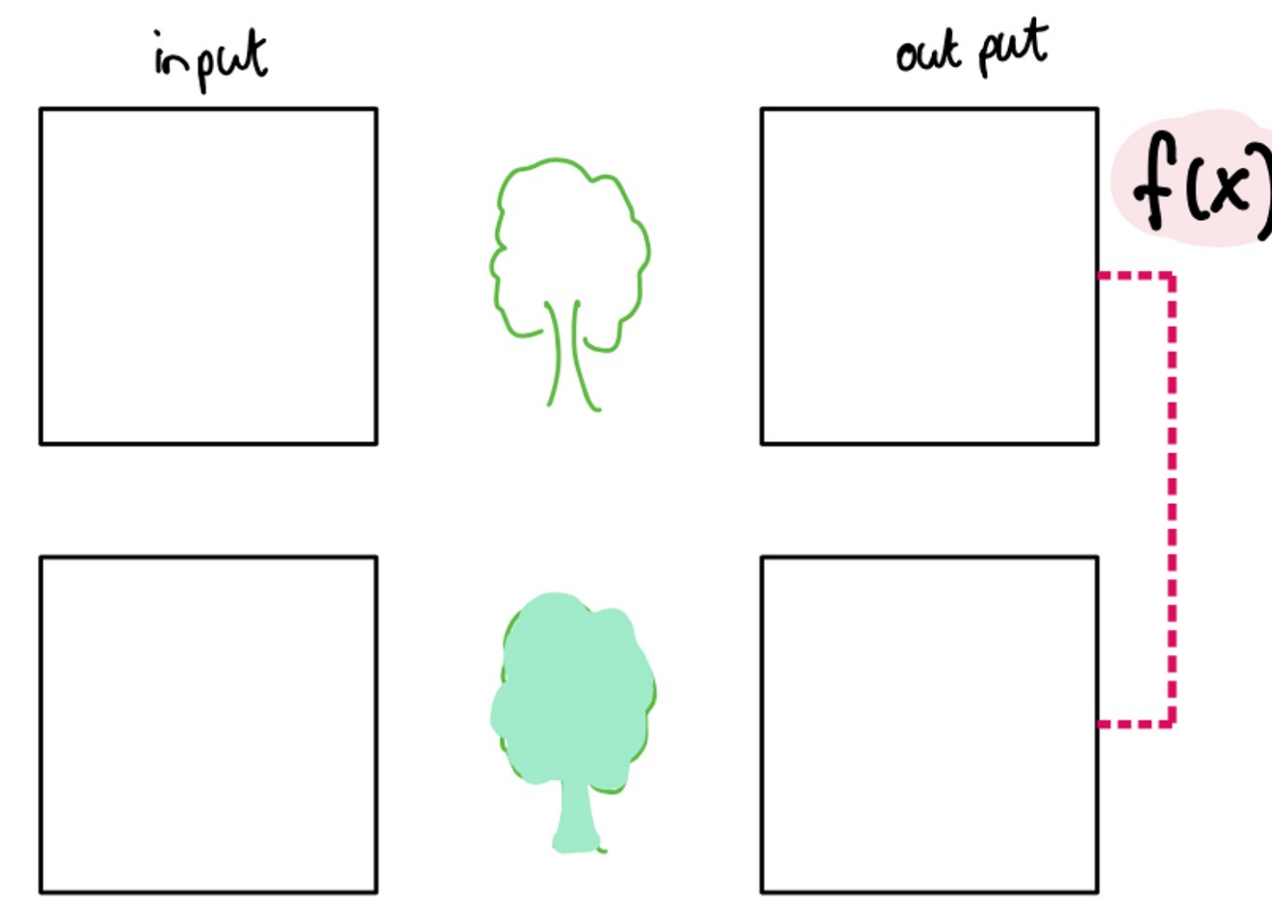
→ No, this is in contrast to images, where permuting the pixels creates a different image, and to text, where permuting the words creates a different sentence.

Node permutation equivariance

Reminder: Transformation equivariance in Euclidean deep learning

A function $f[\mathbf{x}]$ of an image $\mathbf{x}$ is *equivariant* to a transformation $t[\mathbf{x}]$ if:

$$f[t[\mathbf{x}]] = t[f[\mathbf{x}]]$$



- $f[x]$ is equivariant to the transformation $t[x]$ if its output changes in the same way under the transformation as the input.
- Networks for per-pixel image segmentation should be equivariant to transformations.
- If the image is translated, rotated, or flipped, the network $f[x]$ should return a segmentation that has been transformed in the same way.

Node permutation equivariance in GNNs

Considering all nodes in a graph, GCN computation is permutation *equivariant*.

→ each layer must be equivariant with respect to permutations of the node indices.

If we permute the node indices, the node embeddings at each stage will be permuted in the same way. In mathematical terms, if $\mathbf{P}$ is a permutation matrix then we must have:

$$\mathbf{H}_{k+1}\mathbf{P} = \mathbf{F}[\mathbf{H}_k\mathbf{P}, \mathbf{P}^T\mathbf{A}\mathbf{P}, \phi_k]$$

For node classification and edge prediction tasks, the output should also be *equivariant* with respect to permutations of the node indices.

For graph-level tasks, the final layer aggregates information from across the graph so the output is *invariant* to the order of the nodes. In fact, the output layer achieves the following for any permutation matrix $\mathbf{P}$:

$$\begin{aligned} y &= \text{sig} [\beta_K + \omega_K \mathbf{H}_K \mathbf{1} / N] \\ &= \text{sig} [\beta_K + \omega_K \mathbf{H}_K \mathbf{P} \mathbf{1} / N] \end{aligned}$$

😊 For graphs, it is possible to define networks that ensure equivariance or invariance to permutations.

Problem solving

Problem 13.7 Consider the simple GNN layer:

$$\begin{aligned} \mathbf{H}_{k+1} &= \text{GraphLayer}[\mathbf{H}_k, \mathbf{A}] \\ &= \mathbf{a} [\beta_k \mathbf{1}^T + \Omega_k [\mathbf{H}_k; \mathbf{H}_k \mathbf{A}]], \end{aligned} \quad (13.28)$$

where $\mathbf{H}$ is a $D \times N$ matrix containing the N node embeddings in its columns, $\mathbf{A}$ is the $N \times N$ adjacency matrix, β is the bias vector and Ω is the weight matrix. The notation $[\mathbf{H}_k; \mathbf{H}_k \mathbf{A}]$ indicates vertical concatenation of the matrices $\mathbf{H}_k$ and $\mathbf{H}_k \mathbf{A}$. Show that this layer is equivariant to permutations of the node order, so that:

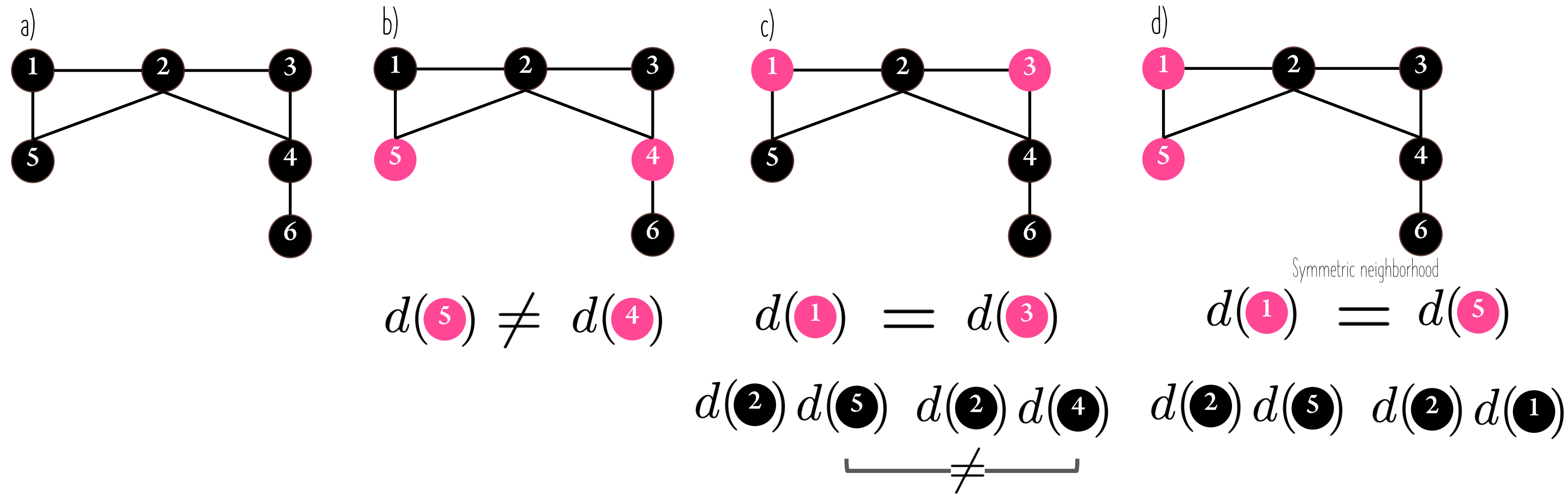
$$\text{GraphLayer}[\mathbf{H}_k, \mathbf{A}]\mathbf{P} = \text{GraphLayer}[\mathbf{H}_k\mathbf{P}, \mathbf{P}^T\mathbf{A}\mathbf{P}], \quad (13.29)$$

where $\mathbf{P}$ is an $N \times N$ permutation matrix.

$$\begin{aligned} \text{GraphLayer}[\mathbf{H}_k\mathbf{P}, \mathbf{P}^T\mathbf{A}\mathbf{P}] &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \begin{bmatrix} \mathbf{H}_k\mathbf{P} \\ \mathbf{H}_k\mathbf{P}\mathbf{P}^T\mathbf{A}\mathbf{P} \end{bmatrix} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \begin{bmatrix} \mathbf{H}_k\mathbf{P} \\ \mathbf{H}_k\mathbf{A}\mathbf{P} \end{bmatrix} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T \mathbf{P} + \Omega_k \begin{bmatrix} \mathbf{H}_k\mathbf{P} \\ \mathbf{H}_k\mathbf{A}\mathbf{P} \end{bmatrix} \right] \\ &= \mathbf{a} \left[\beta_k \mathbf{1}^T + \Omega_k \begin{bmatrix} \mathbf{H}_k \\ \mathbf{H}_k\mathbf{A} \end{bmatrix} \right] \mathbf{P} \\ &= \text{GraphLayer}[\mathbf{H}_k, \mathbf{A}]\mathbf{P} \end{aligned}$$

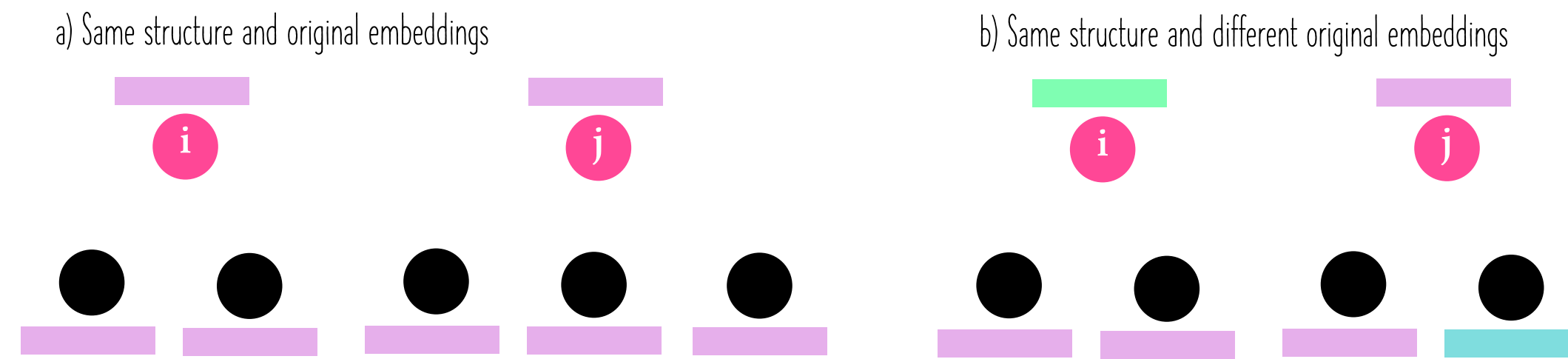
GNN properties: expressive power (expressiveness)

K-hop similarity in node local structure



Expressive power of a GNN model

Definition: An **expressive** GNN model maximizes its **node representation capacity** in the embedding space by mapping **different** local neighborhoods to **different** embeddings (particularly when we have nodes with similar embeddings).



Mathematically, a highly expressive GNN maps local neighborhoods to node embeddings **injectively**.

→ It uses an **injective neighbor aggregation** at each layer k .

→ Since the neighborhood aggregation is modeled through **the node computational graph**, we can define an expressive GNN as a model that maps nodes with **different** computation graphs to different **embeddings**.

For two nodes n and m have two different computational graphs, then their embeddings should be different **at each layer $k+1$** :

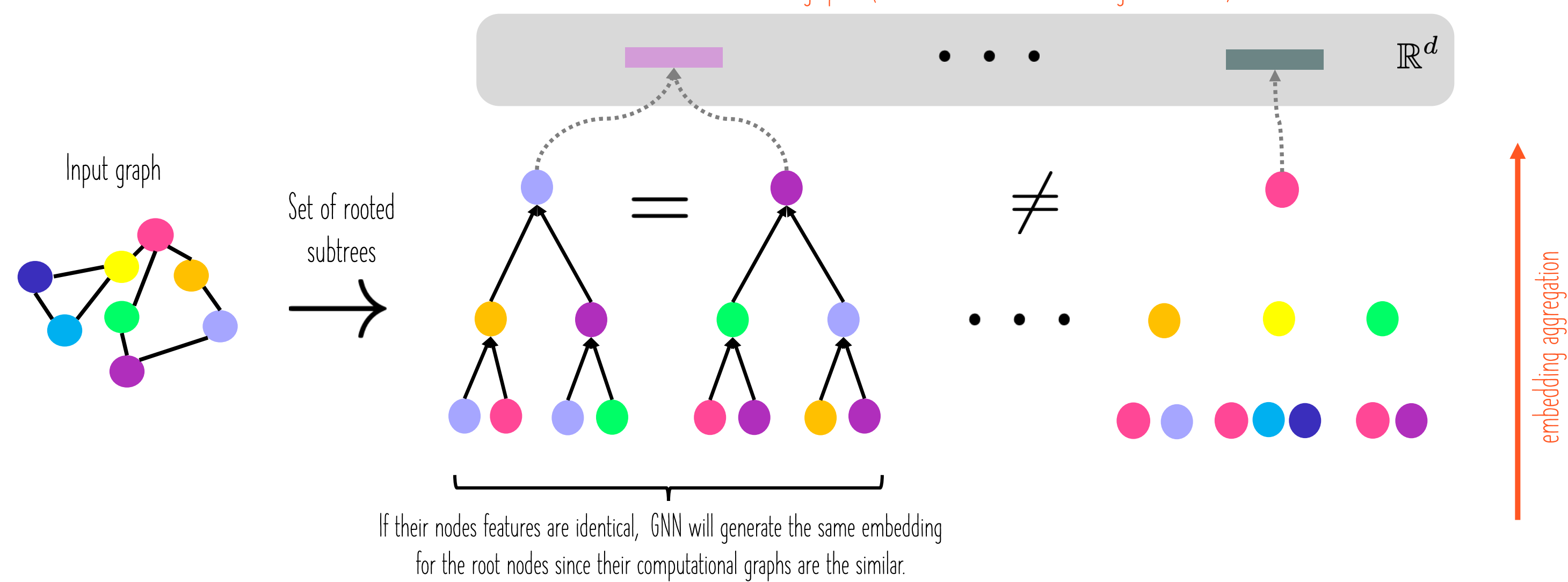
$$\text{If } C(n) \neq C(m) \text{ then } \mathbf{h}_{k+1}^{(n)} \neq \mathbf{h}_{k+1}^{(m)}$$

$$\mathbf{h}_{k+1}^{(n)} = \mathbf{a} \left[\beta_k + \Omega_k \cdot \mathbf{h}_k^{(n)} + \Omega_k \cdot \text{agg}[n, k] \right]$$

Rooted subtrees mapping

Computational graphs are identical to **rooted subtree structures** around each node, which are defined by recursively unfolding neighboring nodes from the root nodes.

embedding space (nodes here have identical original features)



Personal reflections and notes

GNN properties: expressive power (expressiveness)

Personal reflections and notes

Rooted subtree isomorphism test

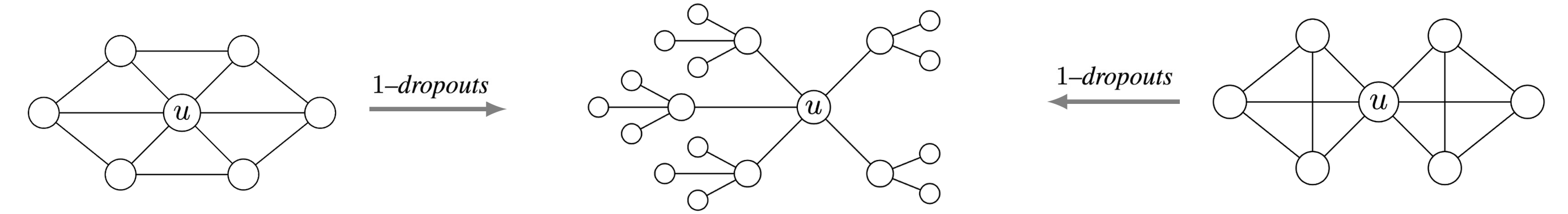
Can GNN node embeddings distinguish different node embeddings neighborhood structure?

→ Yes, if they have been produced using an injective aggregation function in each layer.

Is there a case where GNNs can fail to distinguish different structures?

→ Yes, for instance when two different graph structures produce the same computational graph

1) DropGNN (2021)



2) GIN (2019)

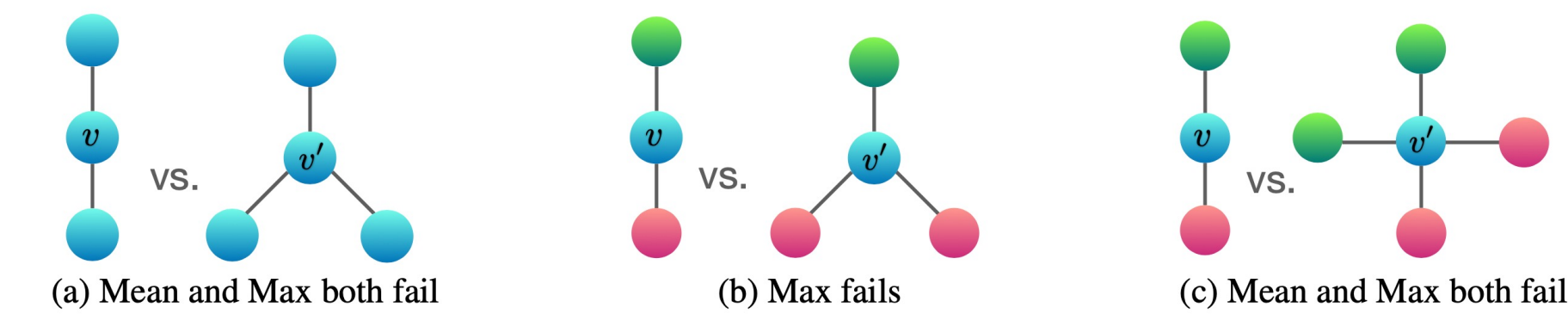


Figure 3: **Examples of graph structures that mean and max aggregators fail to distinguish.** Between the two graphs, nodes v and v' get the same embedding even though their corresponding graph structures differ. Figure 2 gives reasoning about how different aggregators “compress” different multisets and thus fail to distinguish them.

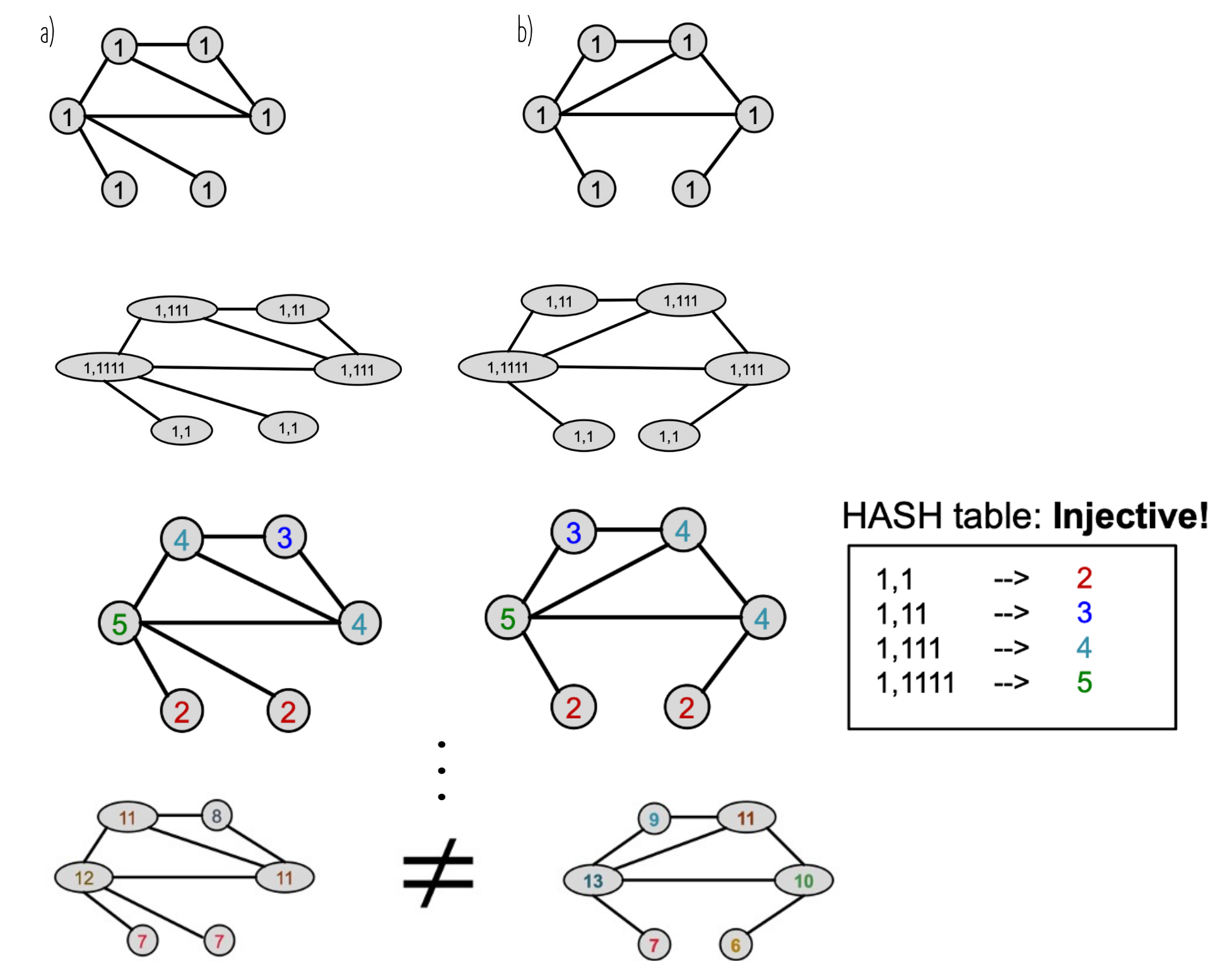
Weisfeiler-Lehman graph isomorphism test

The graph isomorphism problem asks whether two graphs are topologically identical. This is a challenging problem: no polynomial-time algorithm is known for it yet. The Weisfeiler-Lehman (WL) test of graph isomorphism (Weisfeiler & Lehman, 1968) is an effective and computationally efficient test that distinguishes a broad class of graphs.

→ Goal: develop a simple architecture that is provably the most expressive among the class of GNNs and is as powerful as the Weisfeiler-Lehman graph isomorphism test.

WL color refinement algorithm:

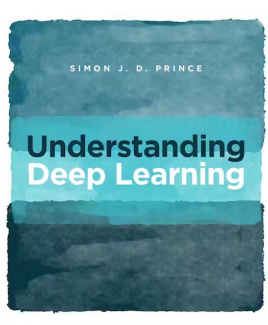
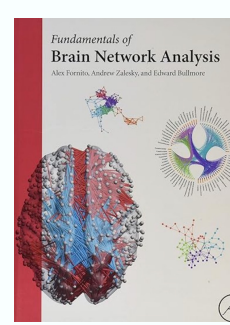
- 1) Assign an initial color $c^0(v)$ to each node v .
- 2) Iteratively refine node color by: $c^{k+1}(v) = \text{HASH}(c^k(v), \{c^k(u)\}_{u \in ne[v]})$, where **HASH** maps different inputs to different colors.
- 3) After K steps of color refinement, $c^k(v)$ summarizes the structure of K -hop neighborhood.



(Courtesy of Jure Leskovec, GNN slides)

References:

1. DropGNN: Papp PA, Martinkus K, Faber L, Wattenhofer R. DropGNN: Random dropouts increase the expressiveness of graph neural networks. *Advances in Neural Information Processing Systems*. 2021;
2. GIN: Xu K, Hu W, Leskovec J, Jegelka S. *How powerful are graph neural networks?* ICLR 2019;
3. WL test: Boris Weisfeiler and AA Lehman. *A reduction of a graph to a canonical form and an algebra arising during this reduction*. Nauchno-Technicheskaya Informatsia, 2(9):12-16, 1968.



GNN properties: expressive power (expressiveness)

Graph Isomorphism Network -GIN- (Xu et al., ICLR 2019)

HOW POWERFUL ARE GRAPH NEURAL NETWORKS?

Keyulu Xu ^{*†}
MIT
keyulu@mit.edu

Weihua Hu ^{*†}
Stanford University
weihuahu@stanford.edu

Jure Leskovec
Stanford University
jure@cs.stanford.edu

Stefanie Jegelka
MIT
stefje@mit.edu

Key results

Our main results are summarized as follows:

- 1) We show that GNNs are *at most* as powerful as the WL test in distinguishing graph structures.
- 2) We establish conditions on the neighbor aggregation and graph readout functions under which the resulting GNN is *as powerful as* the WL test.
- 3) We identify graph structures that cannot be distinguished by popular GNN variants, such as GCN (Kipf & Welling, 2017) and GraphSAGE (Hamilton et al., 2017a), and we precisely characterize the kinds of graph structures such GNN-based models can capture.
- 4) We develop a simple neural architecture, *Graph Isomorphism Network (GIN)*, and show that its discriminative/representational power is equal to the power of the WL test.

Important theorem

Theorem 3. Let $\mathcal{A} : \mathcal{G} \rightarrow \mathbb{R}^d$ be a GNN. With a sufficient number of GNN layers, $\mathcal{A}$ maps any graphs G_1 and G_2 that the Weisfeiler-Lehman test of isomorphism decides as non-isomorphic, to different embeddings if the following conditions hold:

a) $\mathcal{A}$ aggregates and updates node features iteratively with

$$h_v^{(k)} = \phi \left(h_v^{(k-1)}, f \left(\left\{ h_u^{(k-1)} : u \in \mathcal{N}(v) \right\} \right) \right),$$

where the functions f , which operates on multisets, and ϕ are injective.

b) $\mathcal{A}$'s graph-level readout, which operates on the multiset of node features $\{h_v^{(k)}\}$, is injective.

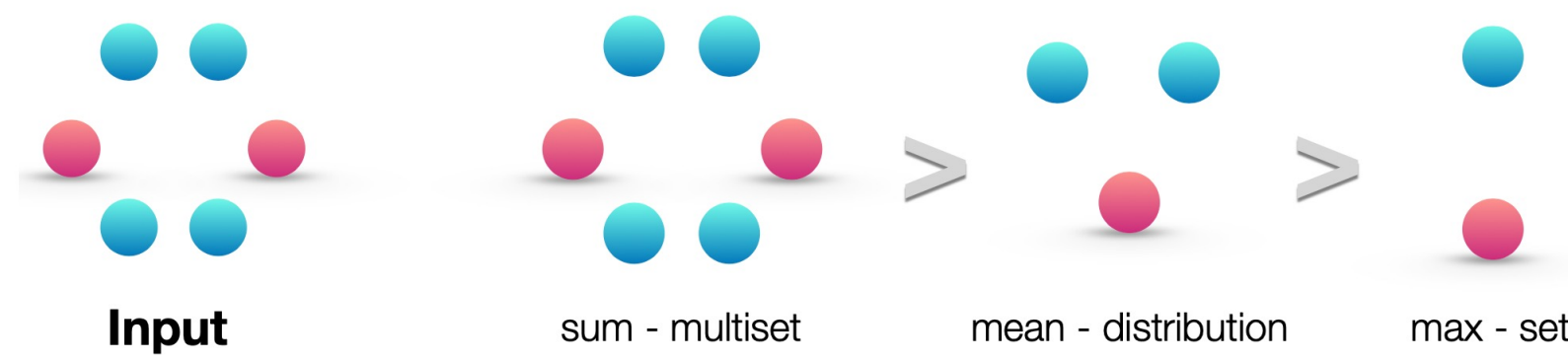


Figure 2: **Ranking by expressive power for sum, mean and max aggregators over a multiset.** Left panel shows the input multiset, i.e., the network neighborhood to be aggregated. The next three panels illustrate the aspects of the multiset a given aggregator is able to capture: sum captures the full multiset, mean captures the proportion/distribution of elements of a given type, and the max aggregator ignores multiplicities (reduces the multiset to a simple set).

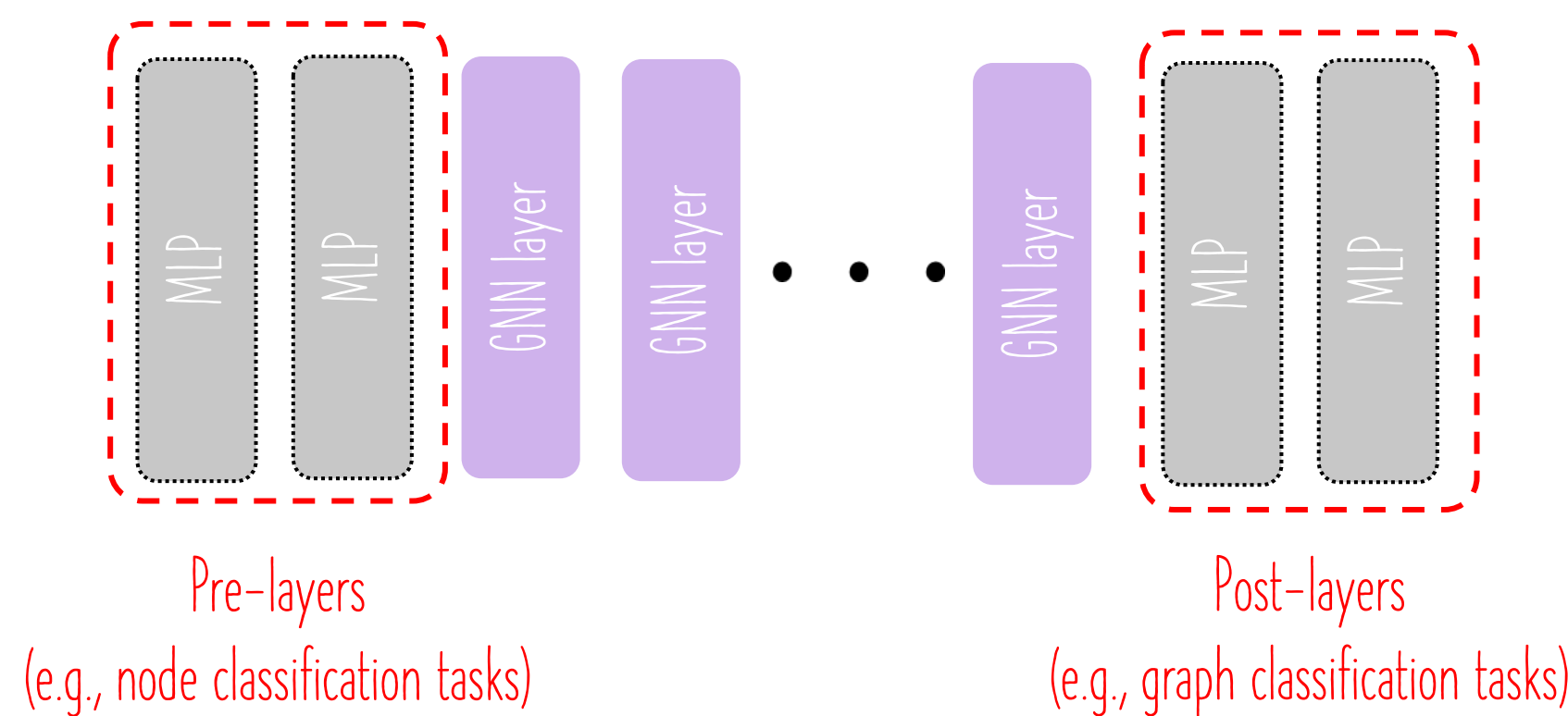
How to maximize the expressiveness of a GNN model?

Multi-layer perceptrons (MLPs) are universal function approximators $\rightarrow$ use them to model and learn f and φ .

GIN is a maximally expressive (powerful) GNN among others that may exist (Xu et al., 2019). Its message update rule takes the following form:

$$h_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right)$$

An **expressive** GNN block can take the following form:



References:

1. DropGNN: Papp PA, Martinkus K, Faber L, Wattenhofer R. DropGNN: Random dropouts increase the expressiveness of graph neural networks. *Advances in Neural Information Processing Systems*. 2021;
2. GIN: Xu K, Hu W, Leskovec J, Jegelka S. How powerful are graph neural networks? ICLR 2019;
3. WL test: Boris Weisfeiler and AA Lehman. *A reduction of a graph to a canonical form and an algebra arising during this reduction*. Nauchno-Tekhnicheskaya Informatsia, 2(9):12-16, 1968.

